# Automatic Feeding, and Monitoring/Control of Water Conditions in an Aquarium

Aquarium system with customizable environmental conditions, feeding schedules, and day/night cycles

Ajtony Pesti, Oliver Albertsen Dominguez, Ricardo Avila, Tuguldur Baatarkhuu

Applied Industrial Electronics, AIE2-5, 2026-06

Semester Project

# Automatic Feeding, and Monitoring/Control of Water Conditions in an Aquarium

Aquarium system with customizable environmental conditions, feeding schedules, and day/night cycles

Ajtony Pesti, Oliver Albertsen Dominguez, Ricardo Avila, Tuguldur Baatarkhuu

Applied Industrial Electronics, AIE2-5, 2026-06

Semester Project

Copyright © Aalborg University 2026

Certain AI tools were used throughout this project for spelling and grammar checking, namely, Grammarly, QuillBot, and LanguageTool. ChatGPT was used for assistance in certain aspects of making the report on Overleaf. Microsoft Copilot assisted in writing the Arduino code and in researching and learning about a variety of topics relevant to this project.

**Electronics and IT**
Aalborg University
http://www.aau.dk

**AALBORG UNIVERSITY**
STUDENT REPORT

**Title:**
Aquarium Management System

**Theme:**
Engineering

**Project Period:**
Spring Semester 2026

**Project Group:**
AIE2-5

**Participant(s):**
Oliver Dominguez
Ajtony Pesti
Ricardo Avila
Tuguldur Baatarkhuu

**Supervisor(s):**
Malte von Benzon

**Copies:** 1

**Page Numbers:** 71

**Date of Completion:**
May 29, 2026

*The content of this report is freely available, but publication (with reference) may only be pursued due to agreement with the author.*

# Abstract

This project explores automation to address the problems associated with manual maintenance of aquariums and fish tanks. In this report, we present our solution to the tedious work of maintaining a tank, a prototype designed to reduce time and effort while improving reliability by reducing human error. There are many existing solutions for this problem, but they often come with high costs, a gap that this project aims to address. The prototype uses sensors and automated controls in order to regulate tank maintenance. This report outlines the research done for this topic as well as the design, development, and testing of the system. The method for addressing the problems involved assembling control and monitoring hardware parts, with the control logic programmed in the Arduino IDE. Hardware components are controlled and monitored across different fish environments and can operate independently. Monitoring data is used by the logic to drive control components, such as the heater, when triggered by a certain limit point, such as temperature. After multiple tests, both dry and in water, we encountered certain obstacles in the code; some have been fixed, and some remain. Moreover, tools such as MATLAB, EasyEDA, Visio, and Visual Studio Code helped illustrate the test results, wiring diagrams, flowcharts, and block diagrams. In general, this project provides an analysis of how a small-scale Arduino setup can be fully utilized for an aquarium management system and offers insight into future work where the solution could be greatly improved with components such as a sensor shield, a PCB, an interface, and IoT integration.

# Contents

**Preface** **vi**

**1 Theoretical analysis** **1**
1.1 Introduction . . . 1
1.2 Flora and nektonic fauna . . . 2
1.2.1 Common animals living in an aquarium . . . 2
1.2.1.1 Siamese fighting fish (*Betta splendens*) . . . 2
1.2.1.2 The guppy (*Poecilia reticulata*) . . . 4
1.2.2 Common plants living in an aquarium . . . 5
1.2.2.1 Amazon Sword (*Echinodorus grisebachii*) . . . 5
1.2.2.2 Anubias (*Anubias barteri*) . . . 6
1.3 Automation . . . 7
1.4 Existing solutions . . . 8
1.4.1 All-in-one controllers: . . . 8
1.4.2 Distributed modular brains: . . . 8
1.4.3 Open integration / DIY IoT stack: . . . 9
1.4.4 Prices . . . 9
1.5 Problem formulation and statement . . . 10
1.6 System requirements . . . 11

**2 Hardware** **12**
2.1 Components . . . 13
2.1.1 Arduino UNO R3 microcontroller . . . 13
2.1.2 DS18B20 Temperature sensor . . . 15
2.1.3 SEN0508 Non-contact flexible liquid level sensor . . . 17
2.1.4 YF-S401 PVC water flow sensor . . . 19
2.1.5 DF9GMS microservo . . . 22
2.1.6 Relay module . . . 24
2.1.7 P25 Submersible heater . . . 26
2.1.8 FIT0709 LED strip . . . 28
2.1.9 FIT0910 DC Water Pump . . . 30

2.1.10 Components list . . . 31
2.2 3D-modeled parts . . . 33
2.2.1 Feeder . . . 33
2.3 Full assembly . . . 33

**3 Software** **35**
3.1 Individually testing components . . . 35
3.2 Final program . . . 40
3.2.1 Considerations . . . 40
3.2.1.1 Non-blocking times . . . 40
3.2.1.2 Relay hysteresis control . . . 40
3.2.2 Arduino program . . . 41
3.2.2.1 Times . . . 42
3.2.2.2 Setup function . . . 44
3.2.2.3 Feeder servo . . . 45
3.2.2.4 Flow sensor . . . 46
3.2.2.5 Temperature sensor . . . 47
3.2.2.6 Relay-heater relation . . . 48
3.2.2.7 LED strip . . . 49
3.2.2.8 Water level sensor . . . 50
3.2.2.9 Water pump . . . 50
3.2.2.10 Environments . . . 51

**4 Testing and discussion** **56**
4.1 Dry testing . . . 56
4.2 Testing with water . . . 57
4.2.1 Plotting graph using MATLAB . . . 60
4.2.2 Discussion . . . 63

**5 Future work** **65**

**6 Conclusion** **68**

**Bibliography** **69**

# Preface

This report was prepared as part of the Applied Industrial Electronics course at Aalborg University during the spring semester of 2026. The project focuses on the monitoring of certain conditions in the tank and their automatic control by a microcontroller. The report is intended for instructors and students within the field of engineering and documents the reasoning behind the report and project, as well as the development process and the final system and its performance.

We would like to thank our supervisor, Malte Von Benzon, for guidance and feedback throughout the project, as well as Jeppe Nørgaard Villadsen, the laboratory technician, for providing access to equipment and facilities, as well as valuable input throughout the semester.

Aalborg University, May 29, 2026

# Chapter 1

# Theoretical analysis

## 1.1 Introduction

Freshwater comprises only around 2.5% of all total water on the planet, of which only 1.2% is surface water, with the vast majority of this freshwater being locked in glaciers or underground. Finally, available surface freshwater represents only about 0.3% of all water on the planet (U.S. Geological Survey, Water Science School). To put these numbers into perspective, the total volume of water on Earth is about 1.386 billion cubic kilometers; from that total volume, only 4,158,000 cubic kilometers is actually available surface freshwater. There currently exist 18,898 recognized freshwater fish, which represent 51% of all fish species on the planet. Freshwater habitats severely lack biodiversity, which ensures resiliency in the ecosystems. Biodiversity in populations of freshwater fishes has been declining at twice the rate of other marine and terrestrial habitats, with one-third of all freshwater fish species being endangered and threatened with extinction, and 80 species already declared extinct. Several threats affect freshwater ecosystems, namely, habitat loss, pollution (from many different sources), over-abstraction of water, over-exploitation of resources, illegal trade of species, the introduction of non-native and invasive species, and loss of river flow due to infrastructure development (IUCN/SSC Freshwater Fish Specialist Group).

Fishkeeping is an extremely prominent hobby around the world, from small children to grown adults. A big reason for this is simply that watching fish swim in an aquarium can calm nerves, relieve stress, and reduce anxiety. This specific hobby requires certain tools, such as an aquarium or fish tank. An aquarium is defined in the Cambridge Dictionary as "a glass container in which fish and other water creatures can be kept" and, on a larger scale, as "a building, usually open to the public, that holds many aquariums." These glass containers, therefore, are tasked with keeping a proper and safe environment in which the species residing in them can live and prosper.

For the fauna and flora to live comfortably and thrive in captivity, certain requirements need to be met. Additionally, different species originate from extremely different habitats with differing conditions, which may lead to incompatibilities inside the tank. Conditions such as temperature, salinity, pH, ammonia levels, water volume, dissolved oxygen, water flow, light levels, and feeding times and schedules must be measured and controlled to ensure a safe and appropriate living environment.

Most readily available aquariums are semi-automated, with some only having monitoring abilities, which means that if the conditions are not optimal for the species contained in them, the environment will require manual alterations to the aforementioned living parameters. For this reason, an automated aquarium management system was developed with the ability to monitor different factors such as temperature, water level, and water flow, and the capability of controlling certain environmental characteristics autonomously based on the different sensors' readings, while also being able to automatically feed the fish, and change brightness inside the aquarium, on a preset schedule.

## 1.2 Flora and nektonic fauna

The life within an aquarium can be divided into two parts. The first one is flora, which includes the plants living in the aquarium; the second one is nektonic fauna, meaning all the animals living in the aquarium. In nature, there are complex processes happening between the flora, the nektonic fauna, and the environment without external help. In an automated aquarium, these processes need to be replicated by hand or via technology. The flora and the nektonic fauna are living in a symbiotic way. The animals consume oxygen and also produce waste such as ammonia. The plants feed on the waste produced by the animals, cleaning the water, and through photosynthesis, they return oxygen to the system. In a healthy and successful aquarium, the waste production by the nektonic fauna does not exceed the cleaning capacity of the flora.

### 1.2.1 Common animals living in an aquarium

To keep the animals happy and healthy, their specific needs must be met for optimal living conditions. It is imperative that this knowledge be acquired before setting up the living conditions for the aquarium environment.

#### 1.2.1.1 Siamese fighting fish (*Betta splendens*)

The Siamese fighting fish (*Betta splendens*), commonly known as the Siamese fighter or locally known as the betta, is a freshwater fish native to Southeast Asia. *Betta splendens* are among the most popular aquarium fish in the world, due to their diverse and colorful morphology and relatively low maintenance. Bettas are well known for being highly

territorial, with males prone to attacking each other whenever housed in the same tank; without a means of escape, this will usually result in the death of one or both fish. Female bettas can also become territorial towards one another in confined spaces. Bettas are exceptionally tolerant of low oxygen levels and poor water quality, owing to their special labyrinth organ, which allows for the intake of surface air. These fish thrive in shallow water with dense vegetation.(Wikipedia Contributors, Siamese Fighting Fish)

The ideal temperature for the betta is around 28° Celsius, since lower temperatures (around 24°C) can lead to biased sex ratios with a higher number of female offspring; on the other hand, higher temperatures (around 33°C) can lead to higher larval mortality rates and a higher number of males. A temperature of 28°C also prevents the spread of certain pathogens, leading to disease prevention. The summer natural light cycle of the fighting fish can be replicated using a 14-10hr light-dark cycle (Lichak et al.).

Good water quality is indispensable when keeping betta, since bad water quality may leave the fish more prone to disease and illness. Ammonia, a byproduct of uneaten food or fish waste, can prove dangerous at levels above 0.25 ppm (parts per million), which is equal to 0.25 mg/L. Additionally, bettas can withstand a pH range of 5 to 9 and salinity levels of 6,000 ppm (6,000 mg/L). Despite this pH range, this condition should be kept at a certain level to create a stable environment, which aims to reduce possible behavioral variations (Lichak et al.).

Feeding the betta dried fish food causes its mating rates to drop by around 70%, and so the fish require sustenance from other foods like brine shrimp, which the adult fish only need to be fed once a day. The daily feeding time should coincide with one of the 14 hours of the day/light cycle in the aquarium. Larval stage and juvenile stage fish receive approximately 20 to 50 shrimp per fish, which is consumed in 5 to 15 minutes, 2 to 4 times a day, whereas fish in the adult stage are fed shrimp and food pellets once a day (Lichak et al.).

**Figure 1:** "Siamese-fighting-fish-bettas-1378308-hero-f459084da1414308accde7e21001906c" by Iluminaticharon is licensed under CC BY-SA 4.0. To view a copy of this license, visit https://creativecommons.org/licenses/by-sa/4.0/?ref=openverse

#### 1.2.1.2 The guppy (*Poecilia reticulata*)

Also known as the millionfish or rainbow fish, it is an animal originating from northeast South America. It has been introduced to many environments and is now found all over the world. Guppies are now one of the world's most widely distributed tropical fish and one of the most popular freshwater aquarium fish species. They are highly adaptable and thrive in many different environmental and ecological conditions. Guppies are used as a model organism in the fields of ecology, evolution, and behavioral studies.

Guppies prefer a hard-water aquarium with a temperature between 25.5 and 27.8 °C. and salt levels equivalent to one tablespoon per 19L. They can withstand levels of salinity up to 1,5 times that of normal seawater, which has led to them being occasionally included in marine tropical community tanks, as well as in freshwater tropical tanks. Guppies are generally peaceful, though nipping behavior is sometimes exhibited between male guppies.(Wikipedia Contributors, The guppy)

The ideal temperature for the guppy's survival rates is around 26°C, whereas the optimal temperature for growth is approximately 30°C, which ultimately leads to worse survival rates. Temperatures above 30°C harm both growth (when compared to 30°C) and on survival rates (when compared to 26°C) (Shah et al.).

The guppy fish can handle up to 30,000 ppm salinity levels (Periyadi et al.) but if the fish are directly exposed to 30,000 ppm, they can only handle these levels for approximately 12 hours, after which the mortality rate of the guppy shoots up to 100%, unless the salinity is slowly and gradually increased from 0 up to even 38,000 ppm of salinity levels in the water throughout 18 weeks. This way, the mortality rate drops from 100 to 20% at 38,000 ppm, at the end of the 18 weeks. The maximum level of salinity in water that the guppy can handle, through direct exposure, is around 5,000 ppm (Pethiyagoda et al.).

**Figure 2:** "The guppy" by Carlos Eduardo Joos is licensed under CC BY 2.0. To view a copy of this license, visit https://creativecommons.org/licenses/by/2.0/?ref=openverse

### 1.2.2 Common plants living in an aquarium

When it comes to plants, the same thing applies as for the animals. To keep them healthy and content, their optimal living conditions must be known and met.

#### 1.2.2.1 Amazon Sword (*Echinodorus grisebachii*)

Echinodorus grisebachii or Echinodorus amazonicus is commonly known as the Amazon sword plant, although other plants are also known under this common name. The aquatic plant is cultivated for and used in ponds and artificial aquatic habitats. It is native to Cuba, Central America, and South America as far south as Brazil and Bolivia. It has been sold under the name Paniculatus. They prefer good light and grow best in a deep, nutrient-rich substrate. Most will grow in variable water conditions, though the majority need tropical or sub-tropical temperature ranges. The larger specimens make magnificent plants for a larger aquarium.(Wikipedia Contributors, Amazon sword)

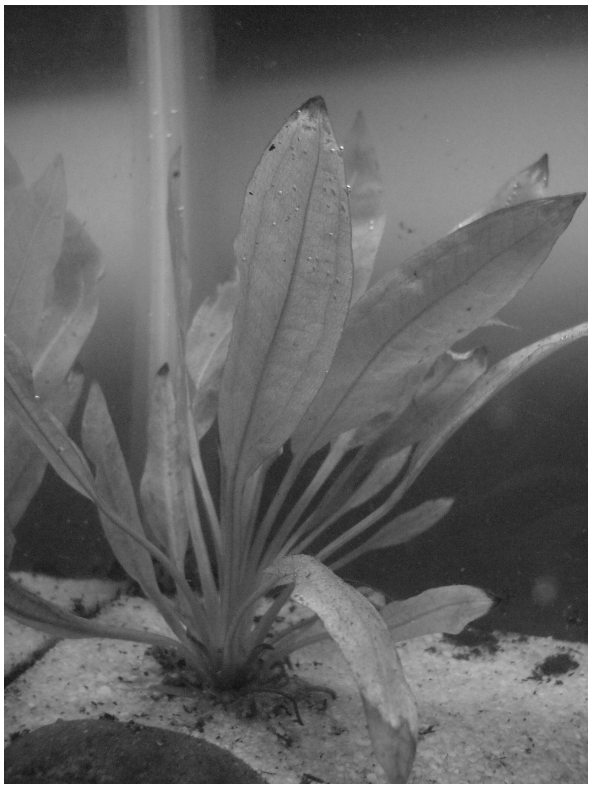


**Figure 3:** "Month 5 - Amazon Sword submersed" by kirsten.elise is licensed under CC BY-SA 2.0. To view a copy of this license, visit https://creativecommons.org/licenses/by-sa/2.0/?ref=openverse

#### 1.2.2.2 Anubias (*Anubias barteri*)

*Anubias barteri* is a West African species of Anubias (see 4), first described in 1860 by Heinrich Wilhelm Schott. It occurs in south-eastern Nigeria, Cameroon, and on Bioko. They primarily grow in rivers and streams but can also be found in marshes. They are characterized by broad, thick, dark leaves that come in many different forms. This plant grows well partially and fully submersed. In strong light, the leaves grow more quickly and remain more compact, but it tolerates a range of lighting. It prefers a temperature range of 22–28 °C (Anubias barteri).

**Figure 4:** "Anubias barteri var. glabra-Jardin botanique de Berlin (2)" by Ji-Elle is licensed under CC BY-SA 4.0. To view a copy of this license, visit https://creativecommons.org/licenses/by-sa/4.0/?ref=openverse

## 1.3 Automation

Automated aquarium systems offer clear advantages over manual maintenance through improved consistency, removal of human error, and by allowing continuous monitoring. When a tank is checked manually, whether it be once every few days or even once a week, all criteria and parameters are checked at that specific time. This allows sudden changes in temperature, pH, or ammonia levels to remain unchecked for long extended periods. An automated system tackles this issue by continuously monitoring the tank's condition and immediately responding if it detects any shifts outside the safe range. Since both fish and aquatic plants are sensitive to environmental conditions, constant monitoring is key to maintaining a stable environment and the long term sustainability of the aquarium biota. Along with continuous monitoring, another major benefit to consider is the elimination of human error. Human error in the context of manual maintenance refers to mistakes, oversights, or inconsistencies made by a person while managing or maintaining

an aquarium that negatively affect the stability of the environment contained within the aquarium, whether it's errors of omission, commission, judgment, timing, measurement, or monitoring.

## 1.4 Existing solutions

Automated aquarium systems can be defined as a full-stack control system that converts sensor signals from temperature, pH, ORP, conductivity/salinity, water levels/leaks, flow, and ammonia/nitrate into actuation decisions by switching AC outlets, dosing food, water pump speed control, lighting control and alarms, where it uses embedded controllers, often with optional cloud/mobile integrations. The most common automated aquarium system architectures are all-in-one controllers, distributed modular systems, and open-source IoT stacks.

### 1.4.1 All-in-one controllers:

Commercial all-in-one controllers combine multi-parameter probing with durable automation/switching and strong expansion interfaces. The system aims to be central to the commanding nodes of sensors-in and output devices-out, using hardware ports and add-on modules to scale. Typical examples are A3 Apex Pro by Neptune Systems, and ProfiLux 4 by GHL. The core idea of this system is using one central controller as the brain, whereas everything else attaches via vendor expansions such as probing sensors, level sensors, flow modules, dosing interfaces and power bars.

**Control stack highlights:**

- Sensors: probe front ends of pH/ORP where it is high impedance and noise-sensitive, or digital/low-speed sensors such as temperature, water level, flow, and leak ropes
- Local controller: a rule engine utilizing schedule and event-driven logic, alarm/alert subsystem, and logging
- Actuation: mains switching, outputs of 0-10V, DC power rails, switch inputs

This standard control stack demonstrates how aquarium sellers combine safety and automation through probe inputs, power monitoring, modular expansion buses, and alert systems.

### 1.4.2 Distributed modular brains:

Distributed modular systems treat each module as a "brain" that can keep operating even with partial communication failure, prioritizing redundancy and solid connectors. A typical example is HYDROS Control X4. The core idea is: rather than one controller being

a single point of failure, multiple modules possess computing/scheduling capability, and the system continues operating through partial communication failures.

**Control stack highlights:**

- Multiple edge nodes:each capable of local schedules/decision-making
- Ruggedized I/O and connectors: can be designed for salt-water environments whereby the enclosure strategy is the main part of the architecture
- Mixed I/O focus: sense ports for level/leak/flow and probe ports for pH/ORP, and furthermore DC drive outputs and 0-10V I/O.

This an example of fault containment in case the system prioritizes reliablity, safety, and redundancy.

### 1.4.3 Open integration / DIY IoT stack:

Typical examples are reef-pi, Reef Angel, and various Home Assistant/ESPHome platform stacks. The core idea revolves around treating the aquarium as a domain inside a general automation platform (Home Assistant) or an open aquarium-dedicated stack (reef-pi). Using common IoT patterns (REST/MQTT, YAML/firmware configs) and building a custom sensor/actuator node.

**Control stack highlights:**

- Sensor nodes: ESPHome reading temperature, level/leak,flow, and any custom sensors you can instrument
- Hub: creating state/event automations, integration and APIs through Home Assistent or aquarium-oriented UI modules such as reef-pi
- Actuation: smart plugs/relays/dimmers (Wi-Fi/Zigbee), PWM of 0-10V via appropriate hardware, dosing/water pump drivers

This prioritizes reproducibility, external data sources, and custom control loops where one can customize the systems' telemetry, logging format, alert rules, test harnesses, and integration APIs.

### 1.4.4 Prices

Pricing varies heavily by regions, bundling, and retailing:

Commercial all-in-one (Apex Pro): around 990USD (listed as bundle)

Distributed modular controller (HYDROS Control X4): around 662USD (with temperature and flow sensor included)

Open-source stacks (reef-pi, HA+ESPHome): overall lower price depending on choice of hardware

| System | Category | Sensors | Actuation I/O | Connectivity |
|---|---|---|---|---|
| A3 Apex Pro | All-in-one | T, pH, ORP, Conductivity, Lvl/leak; Flow via FMM | AC + power monitoring; DC24V; 0-10V | Wi-Fi, `Ethernet` |
| ProfiLux 4 | All-in-one | T, pH, ORP, Conductivity, optional O2/humidity | Powerbars (AC) 0-10V, level ports | Wi-Fi + USB |
| HYDROS Control X4 | Distributed modular brains | T, Lvl/Leak, Flow, pH/ORP via probe ports | DC drive ports; 0-10V I/O | Wi-Fi + cloud app |
| Reef-pi | Open source | Configurable T + extendable analog/digital sensors | Relays, PWM/0-10V via hardware | By host (Pi Wi-Fi/Ethernet) |
| Home Assistant + ESPHome | IoT stack | Very broad via integrations; ESPHome supports many sensors | Smart plugs/relays/dimmers | LAN/Wi-Fi + optional Zigbee/etc. |

**Table 1.1:** Comparison between existing solutions

## 1.5 Problem formulation and statement

Manual aquarium maintenance is time-consuming and prone to human error, often resulting in unstable environmental conditions that can negatively impact aquatic life. Industry-standard aquariums are too expensive to be a solution to the problem and can be designed cheaply to function nearly the same, as noted in the existing solutions. Hence, this raises the question:

**"How can an automated system be designed and evaluated to support stable conditions in a domestic aquarium through automated feeding, temperature control, lighting, and monitoring of water flow whilst being low-cost?"**

## 1.6 System requirements

To delve further into the problem statement, below sub-questions are presented:

- Which environmental conditions are important for maintaining a healthy freshwater aquarium?
- Which aquarium maintenance tasks are most vulnerable to human error?
- Which selected parameters can realistically be monitored or controlled using a low-cost Arduino-based prototype?
- How can sensors and actuators be integrated into a functional aquarium management system?
- How can the prototype be tested to verify that the selected functions operate as intended?

These questions will lead to our solutions regarding the hardware and software, and will be fully addressed in the discussion and conclusion chapters. Moreover, these questions lead to what parts to use for the management systems and their requirements, which can be seen in the table below:

| **Part** | **Requirement statement** | **Verification method** |
|---|---|---|
| Arduino | Arduino should be able to connect I/O pins of all the devices and process the information to the IDE | Test/inspection |
| Temperature sensor | Accurately measure the increasing or decreasing temperature of the water and send signals to the IDE | Test/measurement |
| Water flow sensor | Measure the water flow accurately in L/min | Test/calibration |
| Water level sensor | Detect the water when it is in range | Test |
| Heater + relay | Turn on when the temperature is too low and keep an optimal preset temperature range | Test |
| Feeder (servo) | Rotate 180 degrees within the schedule to drop food pellets | Test/calibration |
| LED light | Turn on and off to imitate day/night cycle of the underwater life | Test |
| Water pump + relay | Work according to given schedule and filter the water | Test/calibration |
| Batteries | Supply power to relay and thus the water pump | Test |

**Table 1.2:** System requirements for the prototype

# Chapter 2

# Hardware

The prototype is an Arduino Uno R3-based system for aquarium automation. The system measures aquatic properties within the aquarium through multiple sensors: water temperature, water level, and water flow. Based on these sensors and a time schedule, the Arduino controls actuators: a servo, a heater, a water pump, and an LED light. Thus, the system operates autonomously and provides status parameters and an interface for personal control.

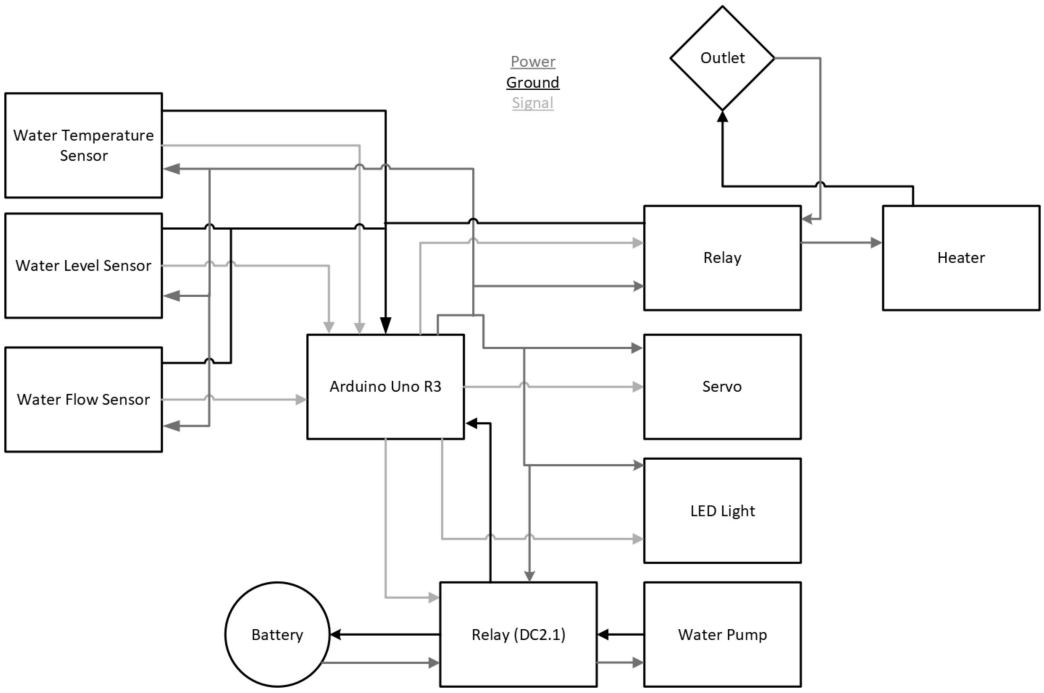



**Figure 5:** Block diagram of the system

As noted in the problem formulation, the system needs to tackle the problem of inconsistent manual maintenance with automation management. Automation in aquarium management is handled commercially by many companies, but it is often implemented with expensive components, a significant gap that this project aims to address. The report will expand on the problem solution in depth, the specific component choices made below:

| **Problem/requirement with cost efficiency in mind** | **Hardware choice** |
|---|---|
| The system needs to coordinate sensors and actuators | Arduino UNO R3 + Breadboard |
| Aquarium temperature must remain stable and adhere to different fish environments | DS18B20 Temperature sensor + Fluval P25 Heater/relay |
| Water should be circulated, oxygenated and monitored in the tank | FIT0910 Water pump + YF-S401 Water flow sensor |
| Water level changes may go unnoticed and harm the hardware through spillage | SEN0508 Water level sensor |
| Feeding can be forgotten or inconsistent | DF9GMS servo (Feeder) |
| Fish and plants need a day/night rhythm | LED light |

**Table 2.1:** Overview of hardware choices

The above are the main hardware choices for the aquarium management system, given the inconsistencies and high-cost issues of modern fish tanks, where fish survival is often at stake with the slightest changes in the environment. The reasoning for each component will be explained further whilst addressing the problem it aims to solve, with respect to the distinction between control and monitoring.

## 2.1 Components

### 2.1.1 Arduino UNO R3 microcontroller

The selected microcontroller was the Arduino UNO R3, which has an ATMega328P processor and 32 kB of flash memory, an input voltage range from the VIN pad between 6 and 20 V, and a maximum input of 5.5 V. With 14 digital I/O pins and 6 analog input pins, it has enough pins to support all the sensors and other devices connected to and

controlled by it. Additionally, the board has a recommended maximum current of 20 mA per I/O pin.

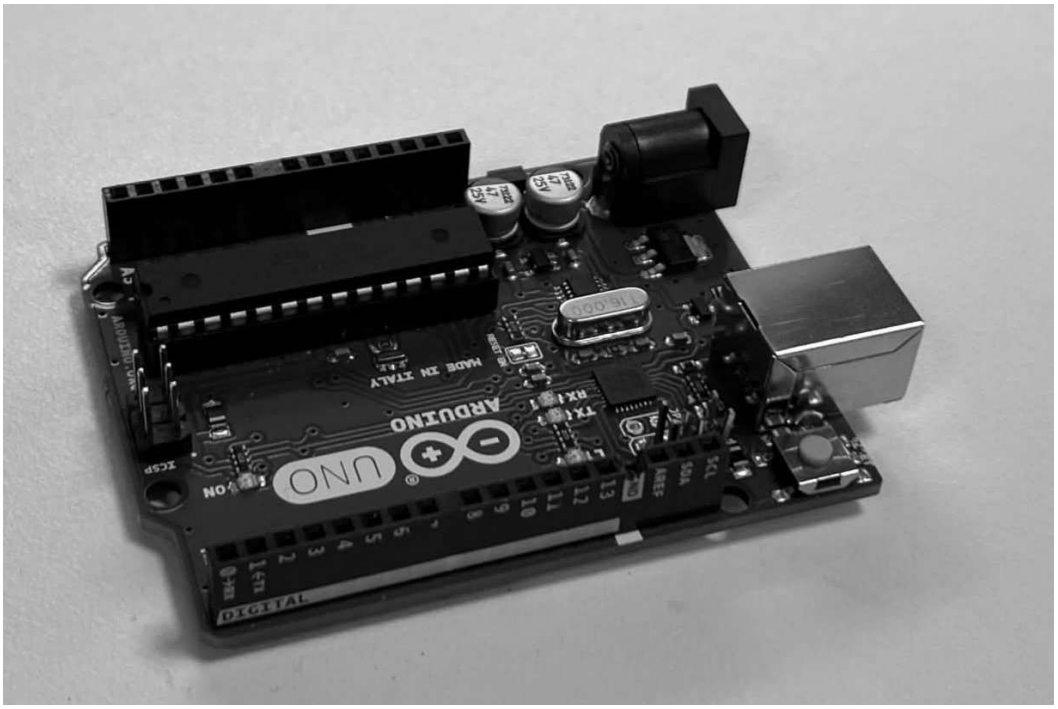



**Figure 6:** Arduino UNO R3 microcontroller

Specifications:

- ATMega328P processor (16 MHz)
- Memory: 2 KB SRAM, 32 KB flash, 1 KB EEPROM
- 6 to 20 V from VIN pad
- 14 digital I/O pins + 6 analog pins
- 20 mA per I/O pin

The Arduino Uno R3 was chosen as the microcontroller because it provides a low-cost but widely supported platform for running an autonomous control system whilst integrating several sensors and actuators. As the brain of the system, the Arduino Uno can acquire measurement signals, execute given control logics, and send output signals to actuate external devices such as the relay, servo, pump, heater, and LED lighting. The choice supports the project goal of reducing inconsistency in domestic aquariums by replacing

manual routines (e.g, remembering feeding and switching lights) with a schedule and sensor signals. Additionally, using a mainstream educational microcontroller reduces the development "cost" in terms of time and risk, since there are many readily available libraries and examples. While the Arduino Uno is resource-constrained compared to modern controllers, it is sufficient for periodic monitoring and simple control strategies and helps keep the overall system within a low-cost budget.

### 2.1.2 DS18B20 Temperature sensor

This sensor is used for monitoring the temperature of the water in the aquarium. The ideal temperature for the guppy is between 25.5 and 27.8 °C (1.2.1.2). The sensor is tasked with making temperature readings, so changes can be made to the water temperature when it deviates from this range.

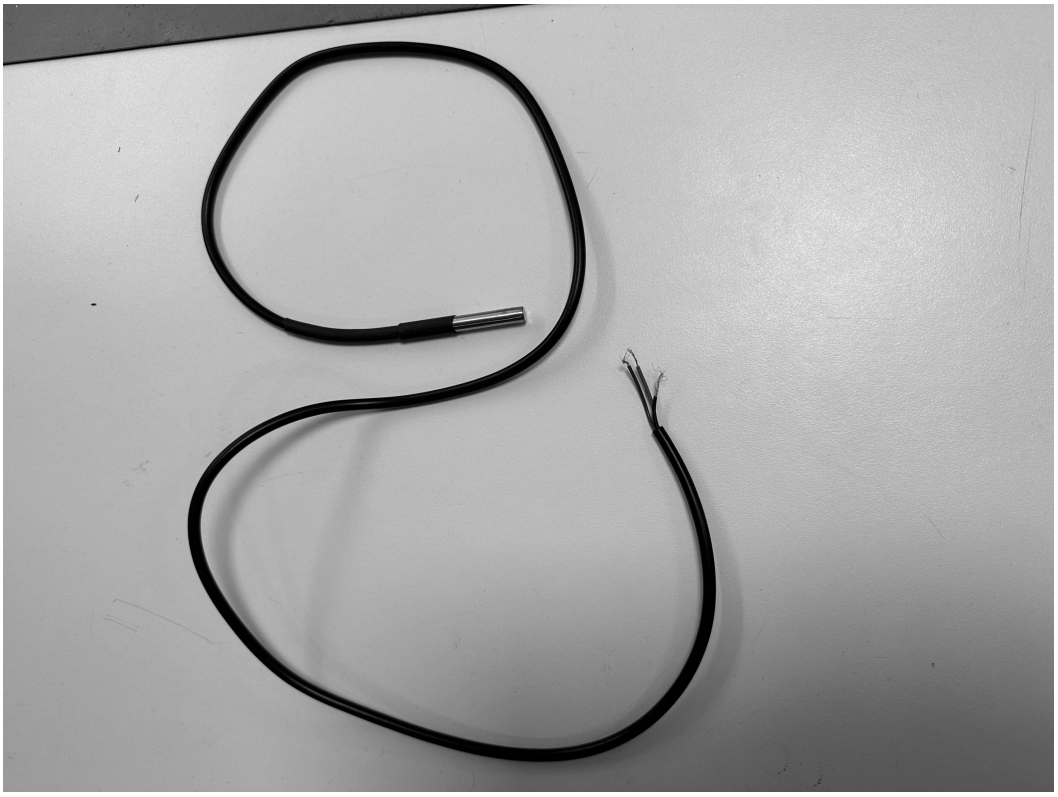


**Figure 7:** DS18B20 Temperature sensor

Specifications:

- 1-Wire interface
- Requires no external components

- Power supply range is 3.0 V to 5.5 V
- Thermometer resolution is programmable from 9 to 12 bits
- Measures temperatures from -55°C to +125°C
- ±0.5°C accuracy from -10°C to +85°C

Pin description:

- GND - Ground
- DQ - Data In/Out
- VDD - Power Supply Voltage

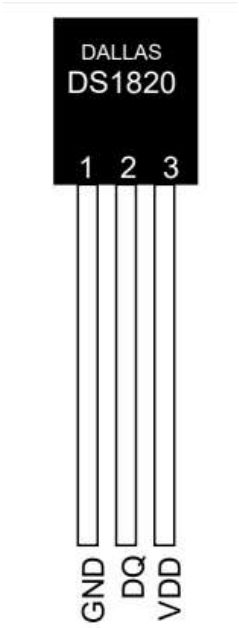



**Figure 8:** Pin arrangement of the temperature sensor

The core functionality of the DS18B20 is its direct-to-digital temperature sensor. The resolution of the DS18B20 is configurable (9, 10, 11, or 12 bits), with 12-bit readings the factory default state. This equates to a temperature resolution of 0.5°C, 0.25°C, 0.125°C, or 0.0625°C. Information is sent to/from the DS18B20 over a 1-Wire interface, so that only one wire (and ground) needs to be connected from a central microprocessor to a DS18B20. Power for reading, writing, and performing temperature conversions can be derived from the data line itself with no need for an external power source. For accurate measurements

a 4.7k resistor should be connected between the VDD and DQ wires. The resistor will pull the signal to a HIGH state ensuring that interference will not ruin the readings.

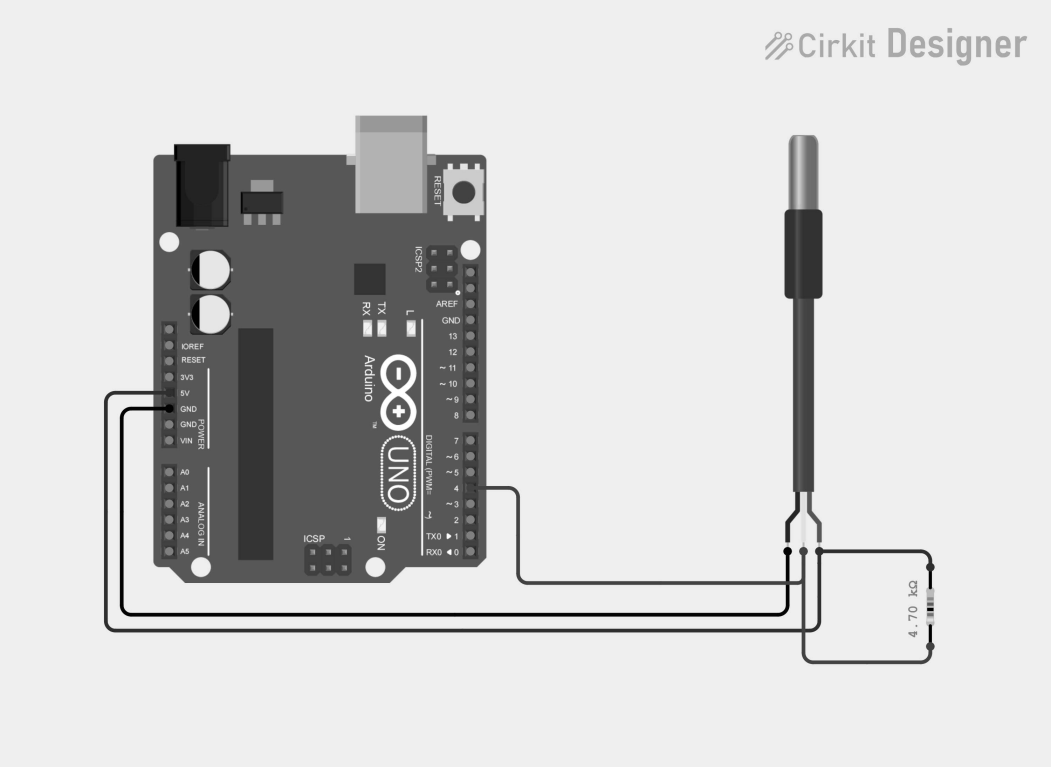



**Figure 9:** Wiring diagram of the temperature sensor with the arduino

The DS18B20 temperature sensor was chosen because water temperature is one of the most essential parameters affecting fish survival and is also one of the easiest to track over time. Unlike pH, conductivity, and salinity probes, this temperature sensor is comparatively stable, low-maintenance, and low-cost. This makes it suitable for a student prototype with a limited budget and time. Furthermore, the DS18B20 uses a digital 1-Wire library, making it easier to use than many analogue sensors that require extra electronics. With the sensor, the system can detect when the water temperature drifts outside the target range, log the temperature for evaluation, and trigger actions via control logic to control the heater. Since aquarium temperature changes relatively slowly, a periodic sampling approach is sufficient.

### 2.1.3 SEN0508 Non-contact flexible liquid level sensor

The liquid level sensor was chosen to address and monitor stability and safety concerns related to abnormal changes in water level, such as overflow risk, or leakage-related droppings. In household aquariums, water level changes can create inconsistent conditions

and lead to water damage in equipment, so monitoring the level provides a meaningful safety function. Although a single-point sensor does not provide continuous level control in centimeters, it is enough for detecting unsafe situations and supporting alarm logic, which fits the project's aim of improving management without being high-cost.

Specifications:

- Working voltages between 2.5 and 5.5 V
- Thickness range of 5 mm
- Output: LOW (liquid detected), HIGH (no liquid detected)
- Working temperature: -30 to 85°C

Pin description:

- GND - Ground
- DQ - Data In/Out
- VDD - Power Supply Voltage

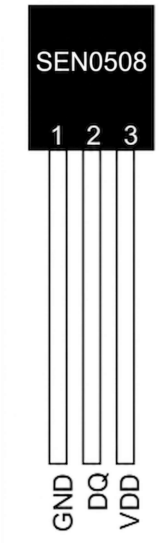



**Figure 10:** Pin arrangement of the water level sensor

The intelligent liquid level sensor uses the sensing capacitance of water to detect the presence of liquid. When there is no liquid approaching the sensor, the sensor has a certain static capacitance due to the distributed capacitance. When the liquid level slowly rises close to the inductor, the parasitic capacitance of the liquid will be coupled to the static capacitor, causing the final capacitance of the inductor to become larger. The changed capacitance signal is then input to the control IC for signal conversion, which will change. The capacitance is converted into the amount of change of an electrical signal, and then a certain algorithm detects and judges the degree of the change. When the amount of change exceeds a certain threshold, the liquid level is considered to reach the sensing point.

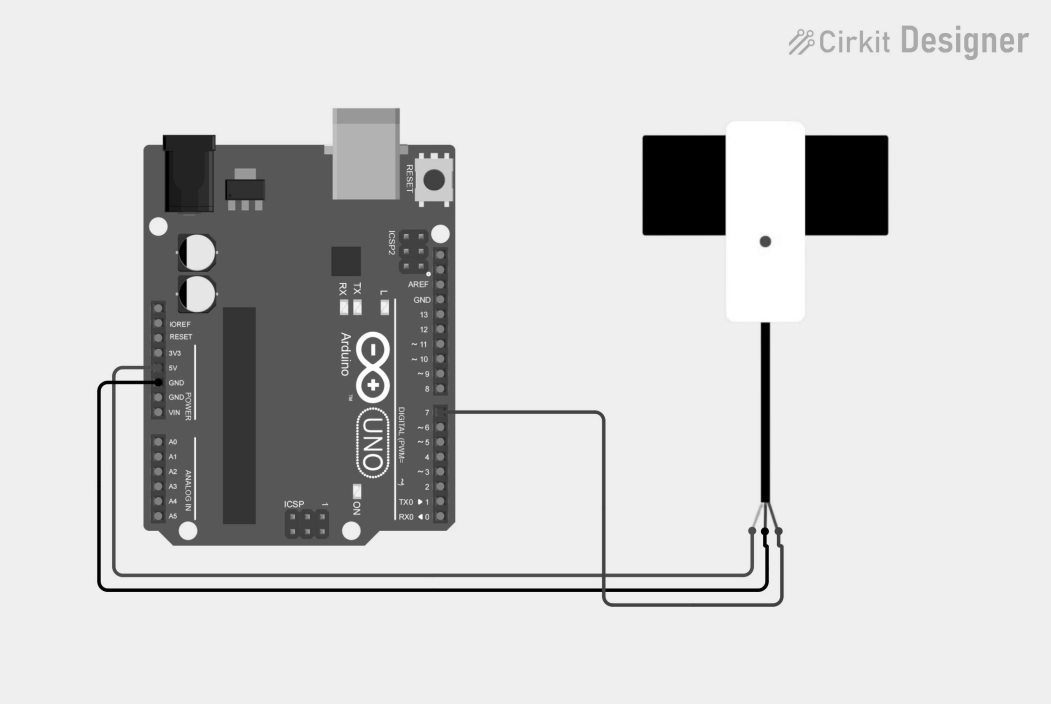



**Figure 11:** Wiring diagram of the water level sensor with the arduino

### 2.1.4 YF-S401 PVC water flow sensor

This sensor is responsible for monitoring and detecting the movement of water within the aquarium. The animals living inside the tank need oxygen dissolved in the water. Stable waterflow plays a key role in keeping the water oxygenated.

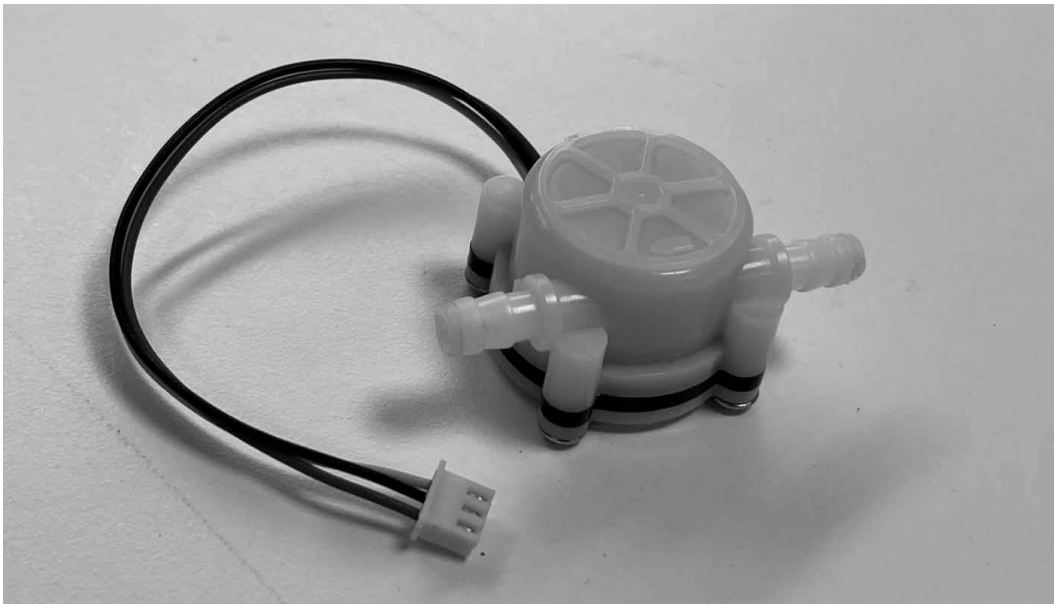


**Figure 12:** YF-S401 PVC Water Flow Sensor

Specifications:

- Working voltage range of 4.5 to 24 V
- Maximum working current is 15 mA
- Flow rate range from 1 to 5 L/min
- Maximum liquid temperature is 120°C
- Maximum water pressure is 1.75 MPa

Pin description:

- GND - Ground
- DQ - Data In/Out
- VDD - Power Supply Voltage

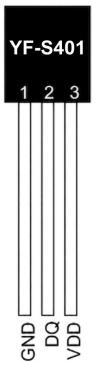



**Figure 13:** Pin assignment of the water flow sensor

The water flow sensor measures the rate of a liquid flowing through it. The YF-S401 water flow sensor consists of a plastic valve body, flow rotor and hall effect sensor. It is usually used at the inlet end to detect the amount of flow. When liquid flows through the sensor, a magnetic rotor will rotate and the rate of rotation will vary with the rate of flow. The hall effect sensor will then output a pulse width signal.

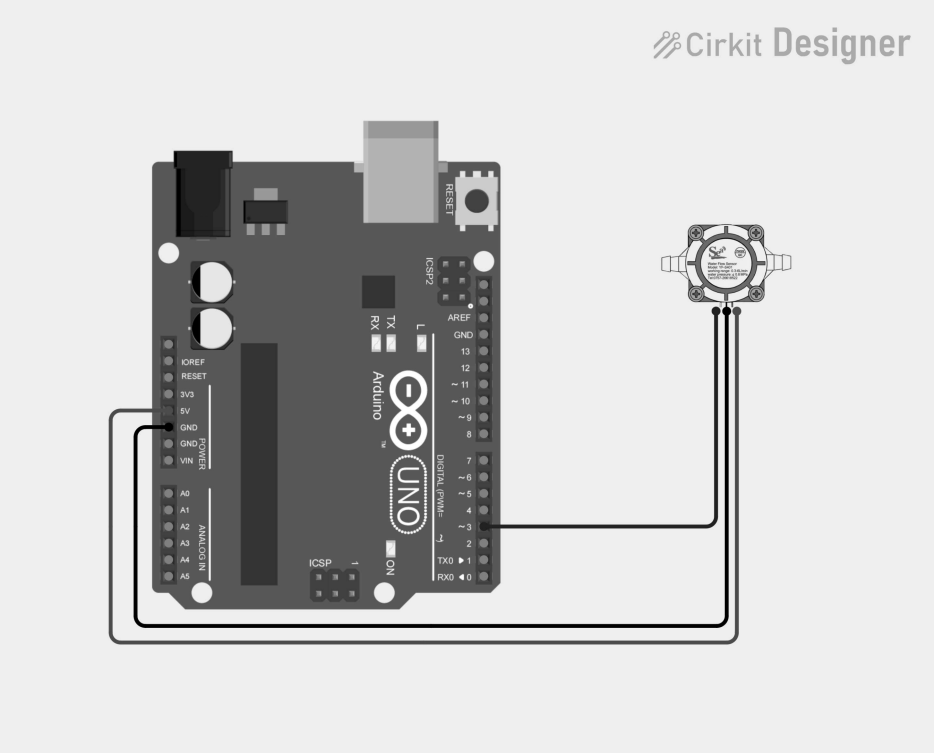



**Figure 14:** Wiring diagram of the water flow sensor with the arduino

The YF-S401 flow sensor was chosen because circulation is essential in oxygen distribution and overall filtration. A reduced flow can indicate issues such as pump malfunction and clogged filters, problems that can develop without being immediately visible to the user. By monitoring flow rate, the system detects the mentioned faults and provides a warning state, improving safety by preventing long operation under uncirculated water. The sensor also helps one understand measurement through timer/counter logic or interrupts, as it outputs pulses proportional to flow. A key limitation is that low-cost flow sensors require calibration and can be sensitive to installation geometry, debris, and air bubbles. This approach still provides valuable functionality while remaining consistent with the low-cost design objective.

### 2.1.5 DF9GMS microservo

The selected control servo is the DF9GMS, which has an operating stall torque of 1.6kg/cm at 4.8V and a 360° rotation angle. This servo will be used to rotate the feeder a set number of times in the span of 24 hours, following a feeding schedule.

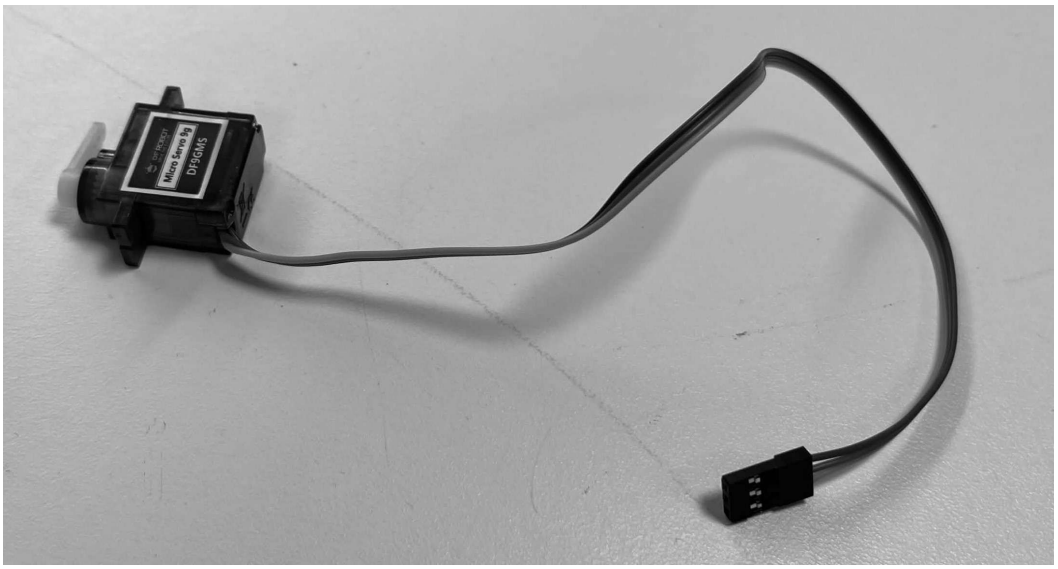



**Figure 15:** DF9GMS servo

Specifications:

- Working voltage range from 4.8 to 6 V
- Working current less than 500 mA
- No load speed is 0.12 seconds/60 degrees (4.8 V)
- Stall torque is 1.6 kg/cm (4.8 V)

Pin description:

- GND - Ground
- VCC - Power Supply Voltage
- PWM - Signal

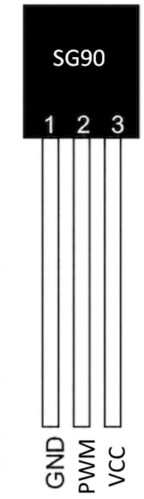



**Figure 16:** Pin assignment of the DF9GMS90 microservo

The DF9GMS is a small but strong servo that can turn 90 degrees in each direction with a torque of 1.6 kg/cm. It operates around 4.8 - 6 V so it is perfect for the Arduino 5 V output pin, and the operating current is less than 500 mA, which works with the 40 mA input/output of the Arduino UNO. A custom 3D-printed container will be attached to the servo so it flips over and drops pellets as the servo does a 180-degree angle change. A timer program will be set in the Arduino IDE which activates the servo 1-2 times every 24 hours. The cons of the servo would be that it is an open-loop system and does not send a signal back to the microcontroller. This means the servo is not able to confirm the exact rotated degree of the arms, but can only receive signals from the microcontroller to change the degree.

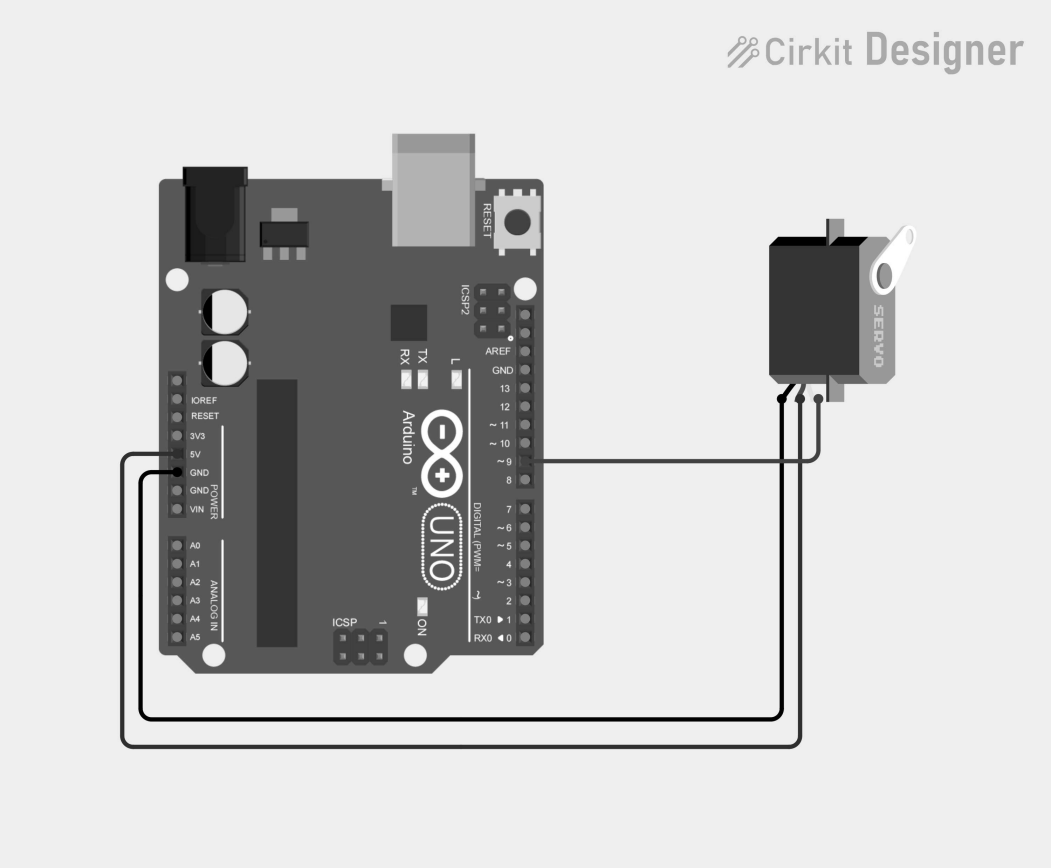



**Figure 17:** Wiring diagram of the servo with the arduino

In general, the microservo was chosen as the actuator for automated feeding because feeding is a common source of inconsistency in household aquariums, since missed feedings and overfeeding can often happen when doing it manually. A microservo provides a low-cost and straightforward solution to scheduled mechanical motion using PWM control. This supports stability by enabling feeding at consistent times and by constraining the feeding portion to the 3D-printed feeder dispenser rather than to user judgment.

### 2.1.6 Relay module

The relay module here plays an important role in circumnavigating the higher operating voltage of the heater 220 - 240 VAC to the lower operating voltage of the Arduino UNO 5 VDC. The relay is a control component and acts as a switch for turning the heater on or off, and the Arduino sends the signal to the IN1 pin (control pin). For example, when the temperature sensor measures less than 22 degrees Celsius, the Arduino sends a signal to the relay to turn on the heater. However, it does not perform as a voltage supply way for

the needed 220 - 240V to the heater, but rather sends the signal to turn on and receive the voltage supply from the connected external supply.

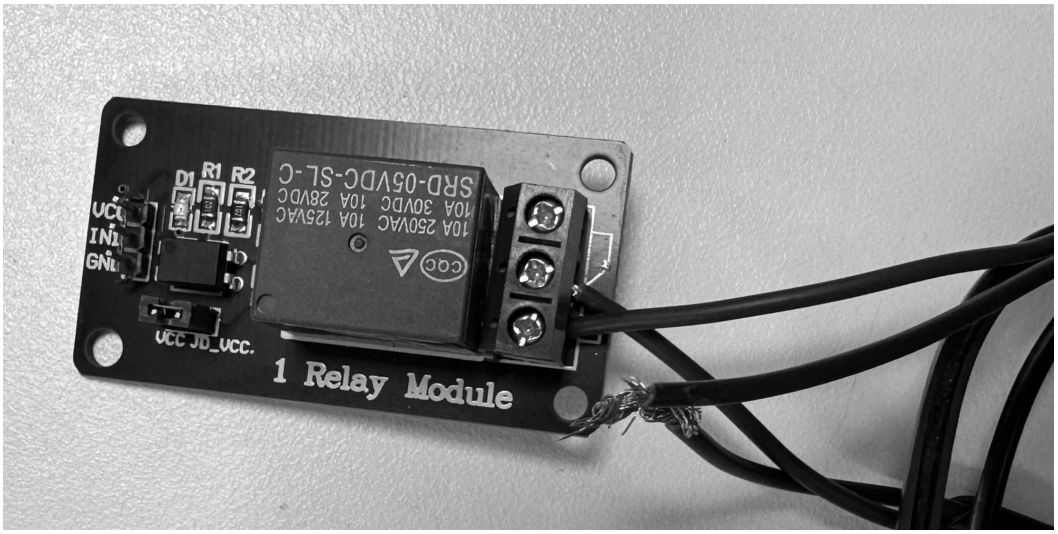



**Figure 18:** Relay module

Specifications:

- Working Voltage is 5 V DC
- Working Current is 70 mA
- Output Capacity is 250 V AC, 10 A or 30 V DC, 10 A

Terminals and pins:

- VCC - Voltage Supply
- IN1 - Control Pin (for signal)
- GND - Ground
- NO - Normal Open Connection
- NC - Normal Close Connection
- COM - Common (for external voltage supply)

This one-channel relay module was chosen to enable the low-voltage microcontroller to safely control higher-voltage loads in an electrically isolated manner. This is relevant if a mains-powered aquarium heater is part of the system. Directly switching such a

load from a microcontroller is extremely unsafe and would exceed the Arduino's electrical limits, whereas the relay provides a clear separation between the low-voltage control area and the high-voltage power area. This design choice supports stable aquarium conditions by creating heater control based on sensor readings and defined preset temperature limits. It is also worth being honest that mains switching increases safety requirements such as enclosure, wiring, and supervision.

### 2.1.7 P25 Submersible heater

The P25 heater is a control component and has an operating power of 25 W and a voltage of 220 - 240 VAC, which is enough to affect a water volume of around 25 L. Moreover, it is rated as IPX8 and can be continuously submerged in water without fear of water leakage. The heater will be attached with two suction cups to the inner wall of the tank. As seen in Figure 18, the heater receives power from an external source and gets a signal from the relay. The relay itself activates from the Arduino with another signal wire.

**Limitations** This heater has a preset temperature range for functionality of 24-26°C. This range cannot be changed, since doing so would require the internal components to be changed/manipulated, which would jeopardize the water sealing and waterproof capabilities of the device.

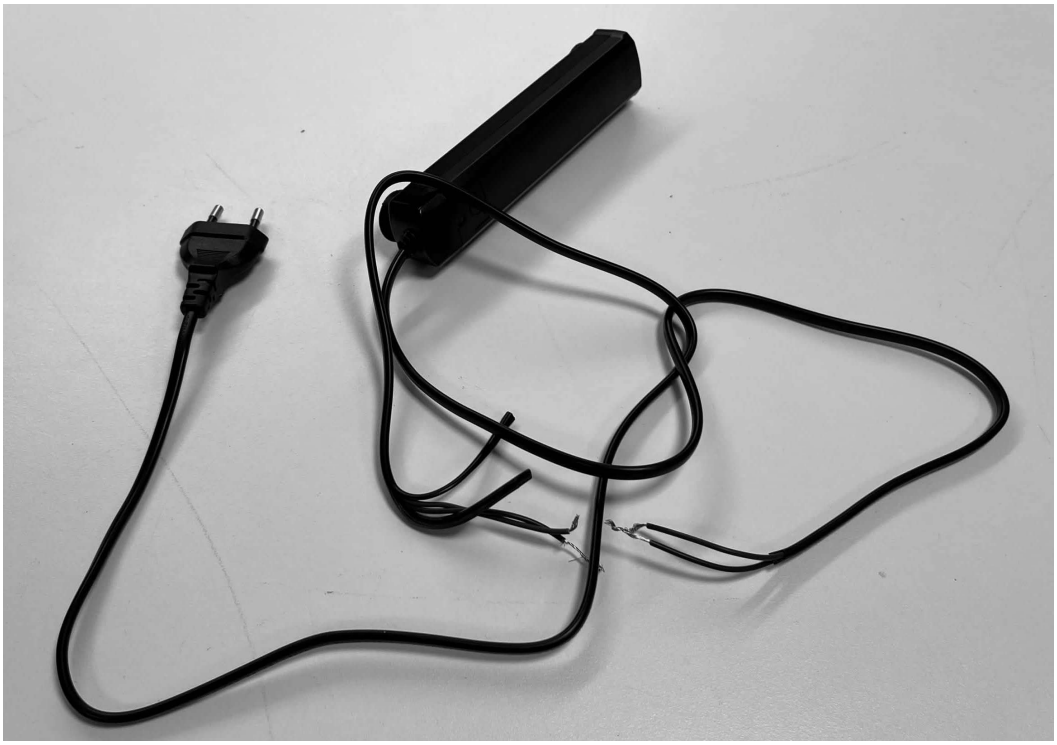


**Figure 19:** Heater

Specifications:

- Working voltage range between 220 and 240 VAC
- Working power is 25 W
- Preset temperature range between 24°C and 26°C
- IPX8 waterproof rating

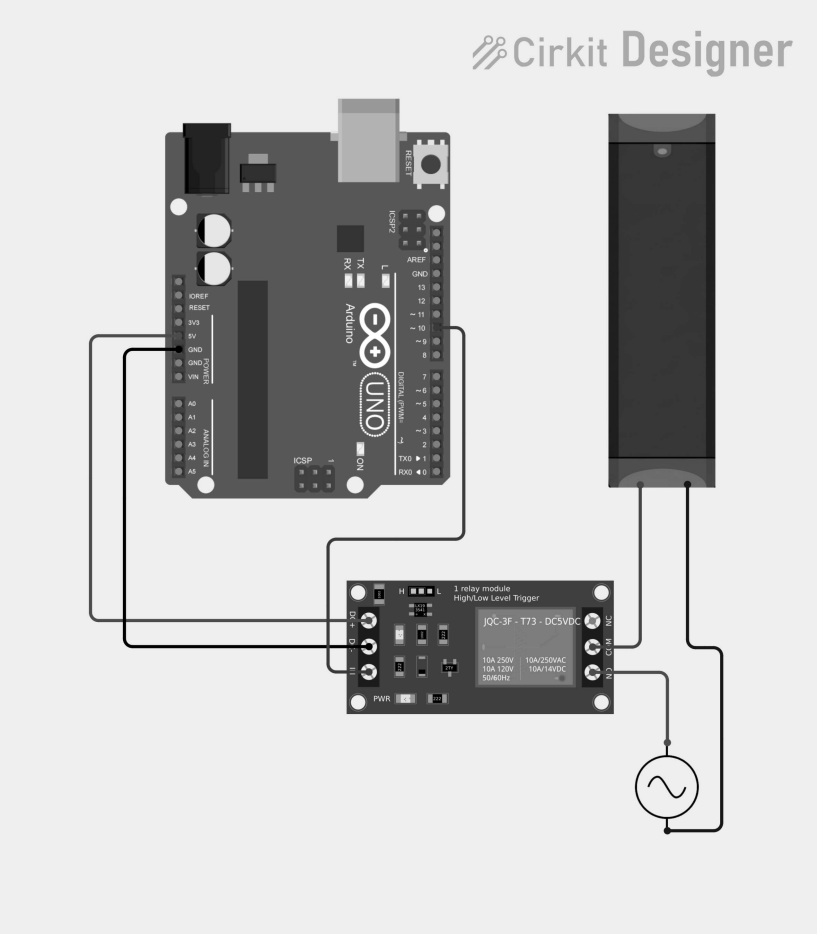



**Figure 20:** Wiring diagram of the heater and relay with the arduino

The P25 Heater is chosen because temperature stability is a strong indicator of a healthy aquarium environment especially in a household setting where room temperature can change. Using a ready-made submersible aquarium heater is a cost-effective choice because waterproofing and basic safety are already handled by a commercial designs, and the project can focus on the monitoring and control system. However, this aquarium heater

also has an inner controller and a thermostat which delivers signals and works via its preset temperature between 24°C and 26°C, which limits overall control from the Arduino Uno R3.

### 2.1.8 FIT0709 LED strip

The LED strip light is a control component and will act as a day/night cycle activator for the fish tank. With a rated power of 6 W and an operating voltage of 5 V, the strip light meets Arduino's operating voltage and emits enough brightness to act as a daylight source. It only has a positive and a negative voltage wire, where Arduino can simply stop or continue the voltage source distributed to the strip light without the need for a signal wire.

Specifications:

- Working voltage of 5 VDC
- Rated Power of 6 W
- Number of LEDs is 160
- Beam Angle of 120
- Length of 50 cm
- 3 M adhesive for mounting

Pin descriptions:

- +5 V - Power Supply Voltage
- GND - Ground

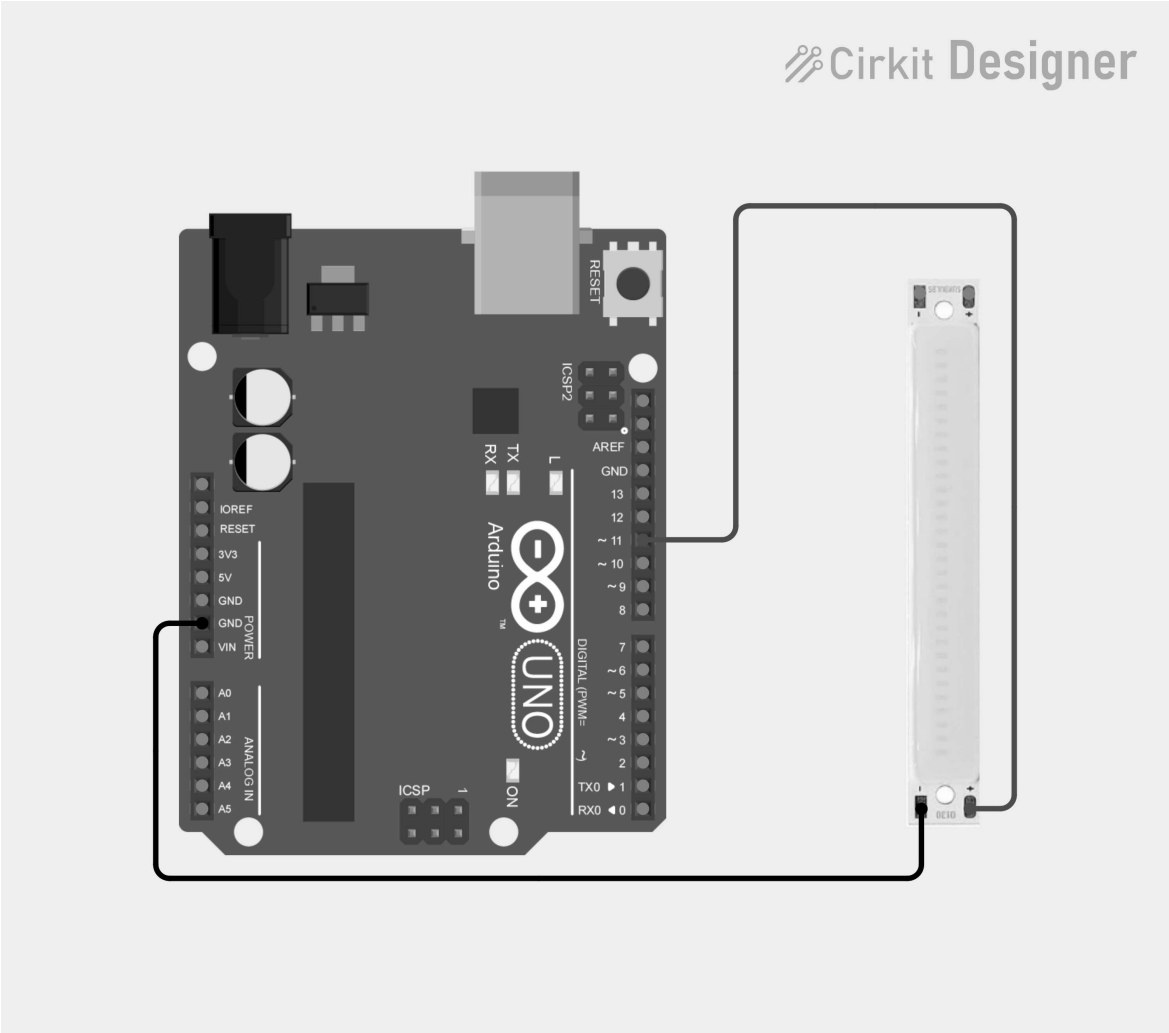



**Figure 21:** Wiring diagram of the LED strip with the arduino

The LED strip light is chosen because a stable day/night cycle matters for fish stress and for plant growth as fish and plants coexist together in a healthy aquarium. In many homes, lighting becomes inconsistent simply because people forget to turn it on or off and so automating the light schedule is a direct way to improve stability. Moreover, the LED light is low-voltage, efficient and easy to mount in a prototype setup while being an actuator that can be demonstrated and tested. The Arduino Uno can control the LED through a simple schedule and potentially dimming, which gives both practical aquarium functionality and a relevant electronic demonstration without the cost of commercial lighting controllers.

### 2.1.9 FIT0910 DC Water Pump

**Figure 22:** DC Water pump image

The chosen DC water pump is a small, low-voltage circulation pump intended for simple aquarium prototyping. It runs on battery power and uses a DC2.1 connector. Due to the higher current requirements, it cannot be directly connected to the microcontroller and so uses an easy relay module to be controlled. The pump, regardless, gives a controllable way to move water and create measurable flow, so you can run repeatable tests through fixed intervals or switch it off in an alarm state.

Specifications:

- Operating voltage: 3-6 VDC

- Current: 150-370 mA
- Flow rate: 80-100 L/h
- Lift/head: 25-45 cm
- Power: 0.4-2 W
- DC2.1 connector
- 1 m wire and 1 m tube

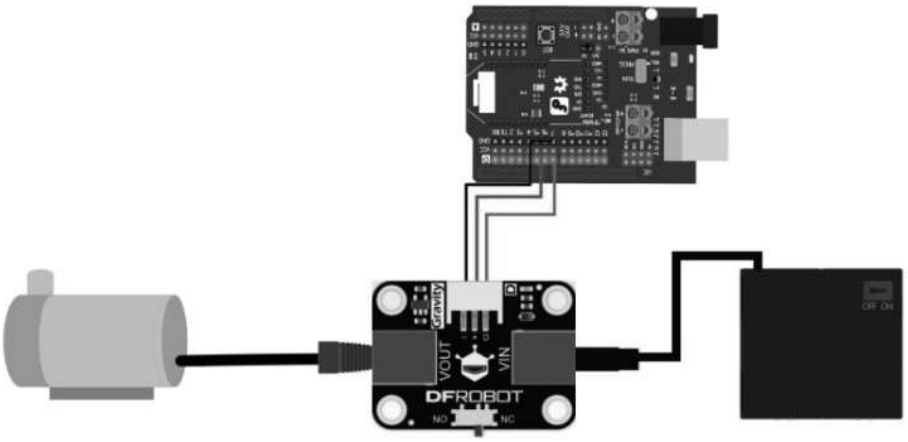



**Figure 23:** Wiring diagram of the water pump with the relay module and batteries

The FIT0910 DC Water pump is a control component to automate water circulation, which helps reduce inconsistency caused by manual actions such as forgetting to run circulation, and/or uneven flow over time. In a household aquarium, stable flow means better filtration and oxygen distribution, so having a controllable pump makes it possible to operate under repeatable conditions and evaluate them in a structured way, such as "pump runs X minutes per hour" or "pump turns off in an alarm state". This pump also fulfills the low-cost criteria because it runs on low voltage (3-6V) and also simple as it is explicitly suggested to be used with easy relay module, making it straightforward to switch on/off from the Arduino without needing a complex motor driver. Additionally, its DC2.1 connector and 1 m cable reduce wiring complexity and make the setup more structured for a prototype.

### 2.1.10 Components list

| Components | Specifications | Function | Price |
|---|---|---|---|
| Arduino UNO R3 | Processor: ATmega328 (16 MHz)<br>Memory: 32 kB<br>Max voltage: 5.5 V<br>Max current/pin: 20 mA<br>Pins: 14 digital, 6 analog | Run the program and control all electronic components | 11.69 USD |
| DS18B20 Tempera-ture Sensor | Supply: 3–5.5 V<br>Range: $-55°$C to $+125°$C<br>Interface: 1-Wire<br>Resolution: 9–12 bits | Measures water temperature and sends data to Arduino | 8.58 USD |
| SES0508 Liquid Level Sensor | Voltage: 2.5–5.5 V<br>Thickness: 5 mm<br>Output: LOW (liquid), HIGH (no liquid)<br>Temp: $-30°$C to $85°$C | Detects water level to prevent overflow | 17.93 USD |
| YF-S401 Water Flow Sensor | Voltage: 4.5–24 V<br>Current: 15 mA<br>Flow: 1–5 L/min<br>Max temp: 120°C<br>Pressure: 1.75 MPa | Monitors water circulation and oxygen distribution | 18.71 USD |
| DF9GMS Microservo | Voltage: 4.8–6 V<br>Current: $<$500 mA<br>Speed: 0.12 s/60°<br>Torque: 1.6 kg·cm | Controls fish feeder rotation on schedule | 7.80 USD |
| Relay Module | Voltage: 5 V DC<br>Current: 70 mA<br>Output: 250 V AC / 30 V DC (10 A) | Switches heater safely without damaging Arduino | 3.90 USD |
| LED Strip Light | Voltage: 5 V DC<br>Power: 6 W<br>LEDs: 160<br>Angle: 120°<br>Length: 50 cm | Provides lighting and day/night cycle simulation | 15.59 USD |
| Heater | Voltage: 220–240 V AC<br>Power: 25 W<br>Temp: $24°$C–$26°$C<br>Rating: IPX8 | Maintains aquarium water temperature | 17.31 USD |
| Water Pump | Voltage: 3–6 V DC<br>Current: 150–370 mA<br>Flow: 80–100 L/h<br>Lift: 25–45 cm<br>Power: 0.4–2 W | Circulates, filters, and oxygenates water | 4.91 USD |

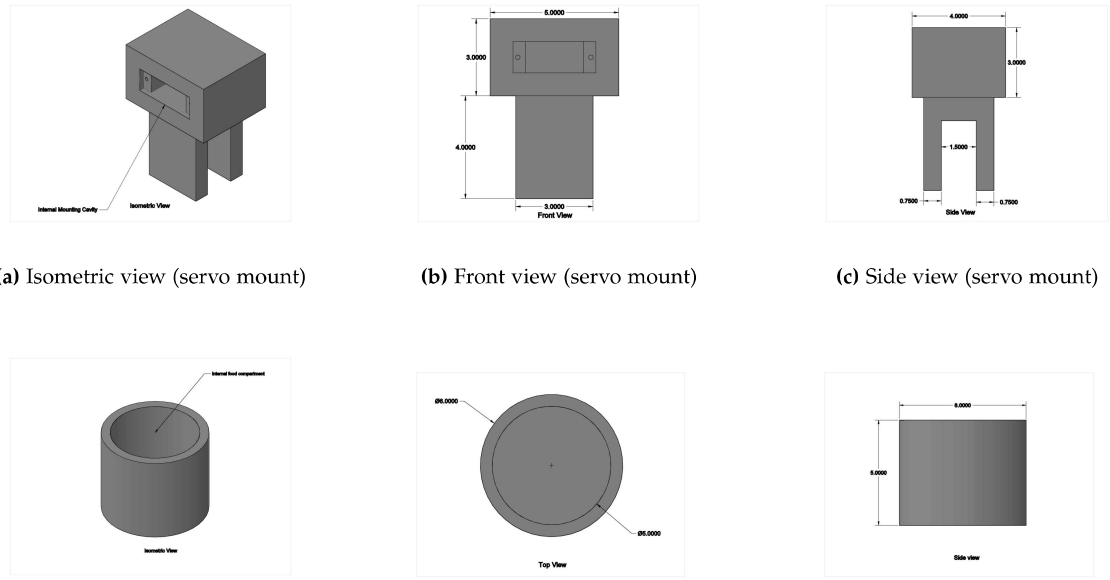


**Table 2.2:** Components list with specifications and functionality

## 2.2 3D-modeled parts

### 2.2.1 Feeder

The servo mount has an opening (check figure 24, a/b) where the servo sits tightly. Additionally, it has two mounting flanges, separated by 3mm (thickness of the fish tank wall), which allow the mount to be placed on top of the tank. The food holder is a cylinder with a large opening, where the fish food is placed. After a set amount of time (3.22), the servo will rotate, causing the cylinder to spin and, consequently, feeding the fish kept inside the aquarium.




**(a)** Isometric view (servo mount) **(b)** Front view (servo mount) **(c)** Side view (servo mount)

**(d)** Isometric view (food holder) **(e)** Top view (food holder) **(f)** Side view (food holder)

**Figure 24:** Views of the micro servo mount and food holder

## 2.3 Full assembly

The final prototype assembly has the LED strip, relay module, temperature sensor, flow sensor, servo, and level sensor connected to the Arduino's digital pins 11, 7, 4, 2, 8, and 12, respectively (see figure 25). All the aforementioned components are connected to the same ground on a breadboard. Connected to that same breadboard is the 5 V power supply coming from the Arduino board, which powers all the components listed, aside from the LED strip, which has its positive wire connected directly to digital pin 11 in the Arduino. The LED strip is connected this way due to the lack of a signal wire.

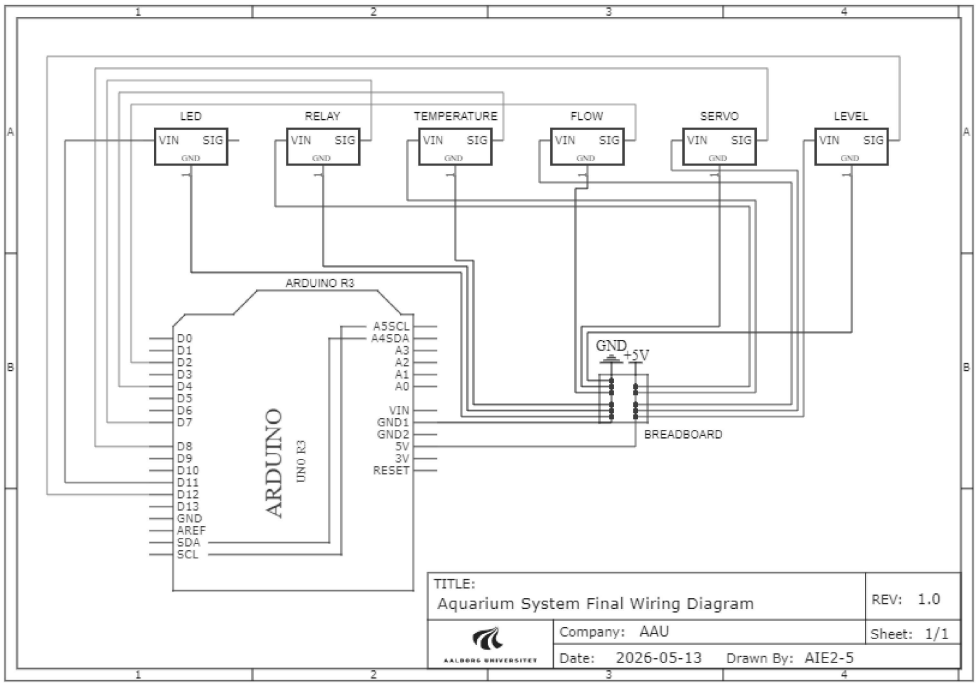



**Figure 25:** Prototype's final wiring diagram

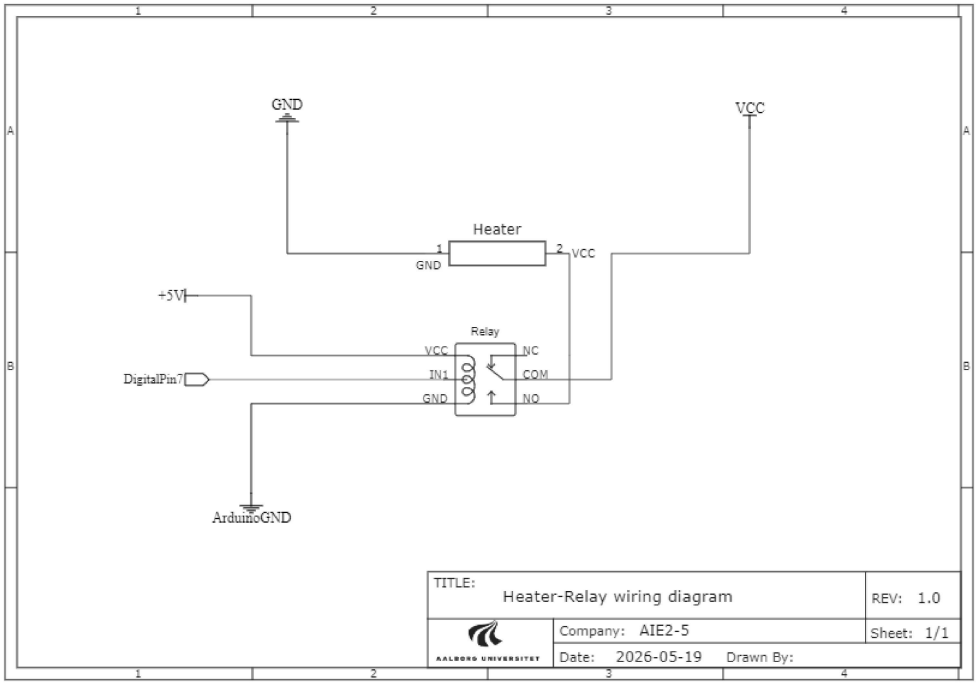



**Figure 26:** Relay-Heater wiring diagram

# Chapter 3

# Software

## 3.1 Individually testing components

**Microcontroller** The Arduino UNO was tested for proper function by running a simple code that made the built-in LED blink.

```
    void setup() {
      // Initialize digital pin LED_BUILTIN as an output.
      pinMode(LED_BUILTIN, OUTPUT);
    }

    void loop() {
      digitalWrite(LED_BUILTIN, HIGH);  // Change state of the LED by
          setting the pin to the HIGH voltage level
      delay(1000);                      // Wait 1 second
      digitalWrite(LED_BUILTIN, LOW);   // Change state of the LED by
          setting the pin to the LOW voltage level
      delay(1000);                      // Wait 1 second
    }
```

**Listing 3.1:** Microcontroller test code

**Temperature sensor** The temperature sensor was tested on air with a code that displays the current temperature read by the device in Celsius and Fahrenheit on the terminal.

```
    #include <OneWire.h> // OneWire library
    #include <DallasTemperature.h> // Dallas temperature library
    #define ONE_WIRE_BUS 4 // Attach pin 4 to the sensor
    OneWire oneWire(ONE_WIRE_BUS);
    DallasTemperature sensors(&oneWire);
```

```
6
7     void setup(void)
8     {
9       Serial.begin(9600); // Set baud rate
10      sensors.begin(); // Start the library
11    }
12
13    void loop(void){
14      sensors.requestTemperatures(); // Request temperature from
            sensor
15
16      Serial.print("Celsius temperature: ");      // Print temperature
             in
17      Serial.print(sensors.getTempCByIndex(0));  // Celsius to the
            terminal
18      Serial.print(" - Fahrenheit temperature: "); // Print
            temperature in
19      Serial.println(sensors.getTempFByIndex(0));  // Fahrenheit to
            the terminal
20      delay(1000); // Wait 1 second
21    }
```

**Listing 3.2:** Temperature sensor test code

**Water level sensor** To test the SEN0508 water level sensor for basic functionality and responsiveness, a code was written, and the program ran under dry conditions. The program takes the output from the sensor, and the built-in LED turns off when in contact with a container containing water.

```
1     const int sensorPin = 2; // Attach pin 2 to the water level
          sensor
2     const int ledPin = 13;     // Onboard LED for status
3
4     void setup() {
5       pinMode(sensorPin, INPUT);  // Initialize the sensor pin as
            input
6       pinMode(ledPin, OUTPUT);     // Initialize the LED pin as output
7       Serial.begin(9600);          // Set baud rate
8     }
9
10    void loop() {
11      int sensorState = digitalRead(sensorPin); // Read the digital
            value
12
13      if (sensorState == LOW) { // Liquid detected
```

```
14          digitalWrite(ledPin, HIGH);
15          Serial.println("Status: [ LIQUID DETECTED ]");
16        } else { // No liquid detected
17          digitalWrite(ledPin, LOW);
18          Serial.println("Status: No liquid...");
19        }
20
21        delay(100); // Wait 100 milliseconds
22      }
```

**Listing 3.3:** Water level sensor test code

**Water flow sensor** The YF-S401 water flow sensor was simply tested for the pulse output and calculations of the flow of water, or in this case, air. The program reads the pulse outputs from the sensor and calculates the flow rate in liters per minute.

```
1       int flowPin = 2; // Attach pin 2 to the flow sensor
2       double flowRate; // Value to be calculated
3       double flowRate1;
4       volatile int count; // Pulse counter integer
5
6       void setup() {
7         pinMode(flowPin, INPUT); // Sets flowPin as an input
8         attachInterrupt(0, Flow, RISING); // Configures interrupt 0 (
              pin 2 on the Arduino Uno) to run the function "Flow"
9         Serial.begin(9600); // Set baud rate
10      }
11
12        void loop() {
13        count = 0; // Reset the counter to start counting from 0 again
14        interrupts(); // Enables interrupts on the Arduino
15        delay (1000); // Wait 1 second
16        noInterrupts(); // Disable the interrupts on the Arduino
17
18        flowRate1 = (count * 2.25); // Take counted pulses in the last
              second and multiply by 2.25mL()
19
20        flowRate = flowRate1 * 60; // Convert seconds to minutes,
              giving mL / Minute
21        flowRate = flowRate / 1000; // Convert mL to Liters, giving
              Liters / Minute
22        Serial.print("Flow rate = ");
23        Serial.println(flowRate); // Print flowRate to Serial
24      }
25
```

```
26      void Flow()
27      {
28        count++; // Increment "count" by 1, when called
29      }
```

**Listing 3.4:** Water flow sensor test code

**Servo** The SG90 micro servo was tested for movement and to verify whether the component worked properly or not. For this purpose, a basic code was written and executed by the Arduino UNO. The code was set to run a simple program that changes the angle of the servo between 10 and 180 degrees every 1000 milliseconds.

```
1       #include <Servo.h> // Servo library
2       Servo servo; // Constant for the servo
3
4       void setup() {
5         servo.attach(8); // Attach pin 8 to the servo
6         servo.write(10); // Initialize at 10 degrees
7       }
8
9       void loop()
10      {
11        servo.write(10); // Rotate to 10 degrees
12        delay(1000); // Wait 1 second
13        servo.write(180); // Rotate to 180 degrees
14        delay(1000); // Wait 1 second
15      }
```

**Listing 3.5:** Servo test code

The servo used in the final prototype was changed due to reasons explained in chapter 4. And so, the previous code snippet 3.5 is the test for the original servo. This original microservo is positional, which fits the design better. The final servo logic in the Arduino program is different because the replacement servo is continuous and directional (check chapter 4).

**Relay module** The 1-channel relay module was tested with a simple code that activates the relay switch every second. A clicking noise is heard from the switch every second.

```
1       #define RELAY_PIN 10 // Attach pin 10 to the relay
2
3       void setup() {
```

```
    pinMode(RELAY_PIN, OUTPUT); // Set pin as an output
  }

  void loop() {
    digitalWrite(RELAY_PIN, LOW); // Turn relay ON
    delay(1000); // Wait 1 second

    digitalWrite(RELAY_PIN, HIGH); // Turn relay OFF
    delay(1000); // Wait 1 second
  }
```

**Listing 3.6:** Relay module test code

**LED strip** The LED strip was tested with a code that switches the brightness level from minimum (off) to maximum (on) every 5 seconds.

```
const int ledPin = 10;  // Attach pin 10 to the LED strip
int brightness = 255;   // 0 - 255

void setup() {
  pinMode(ledPin, OUTPUT);
}

void loop() {
  analogWrite(ledPin, 255); // Set to maximum brightness
  delay(5000); // Wait 5 seconds
  analogWrite(ledPin, 0);
  delay(5000); // Wait 5 seconds
}
```

**Listing 3.7:** LED strip test code

**Water pump** The water pump was tested with a simple code, set to turn the pump on for 5 seconds and then off for 10 seconds.

```
const int pumpPin = 13;

void setup() {
  pinMode(pumpPin, OUTPUT);
  digitalWrite(pumpPin, LOW);   // Pump OFF
}

void loop() {
  digitalWrite(pumpPin, HIGH);  // Pump ON
```

```
10      delay(5000);                    // Run 5 seconds
11
12      digitalWrite(pumpPin, LOW);    // Pump OFF
13      delay(10000);                   // Wait 10 seconds
14    }
```

**Listing 3.8:** Water pump test code

## 3.2 Final program

### 3.2.1 Considerations

#### 3.2.1.1 Non-blocking times

The final Arduino code is intended to have the program's loop run continuously, without stopping, and all the sensors should work and output data at the same time. For this reason, delay() cannot be used in the final code, since it would stop the program until a certain set amount of time had passed. Non-blocking times essentially check whether a certain amount of time has passed or not; instead of waiting, this way, the pausing issue caused by delay() is solved.

$$CurrentTime - LastActionTime > SetTimeForAction \tag{3.1}$$

*CurrentTime* is the current run time of the program,
*LastActionTime* is the last time the action occurred (set to 0 at initialization),
*SetTimeForAction* is a set amount of time until an action happens

If the previous time verification (3.1) proves to be true at any time during the running of the program, for any of the non-blocking times throughout the code, a certain action will happen.

#### 3.2.1.2 Relay hysteresis control

To prevent chatter from the relay module, a range of temperatures needs to be set instead of having a set temperature where the relay turns on and off. It is required to be this way to prevent the relay from constantly turning on and off.

If the temperature is unstable, for example, the set temperature for the relay to turn on or off is 25°C, and the temperature sensor constantly reads between 24.9°C and 25.1°C, the relay would be stuck in an on and off loop, which could damage the component. By setting a range in place, the relay is only either on or off whenever the temperature read

by the sensor remains within that same range, saving the relay from constantly changing states and only changing whenever one of the extremes is reached.

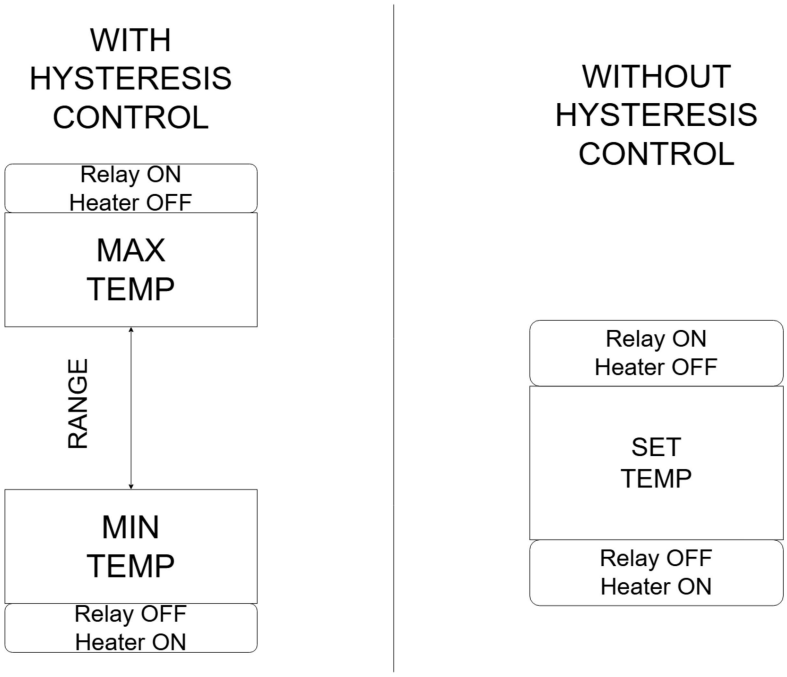



**Figure 27:** Relay with and without hysteresis control

### 3.2.2 Arduino program

Figure 28 is the flow chart of the final program, which visually displays the whole Arduino program and how it functions. When the program starts, the default environment is set (3.20), and all the setup code runs (3.11). The program relies heavily on non-blocking times (see 3.2.1.1), and in each loop, it verifies the times for the sensors, for the feeder servo, for the LED strip, and for the water pump to perform an action. If the time for a certain action to happen is verified, the action occurs; for example, the servo rotates, the sensors output readings, the LED strip turns off, or the water pump pumps water. After any of the actions take place, the times set for each action's verification are reset for the next loop's verifications by saving the last time that the action occurred.

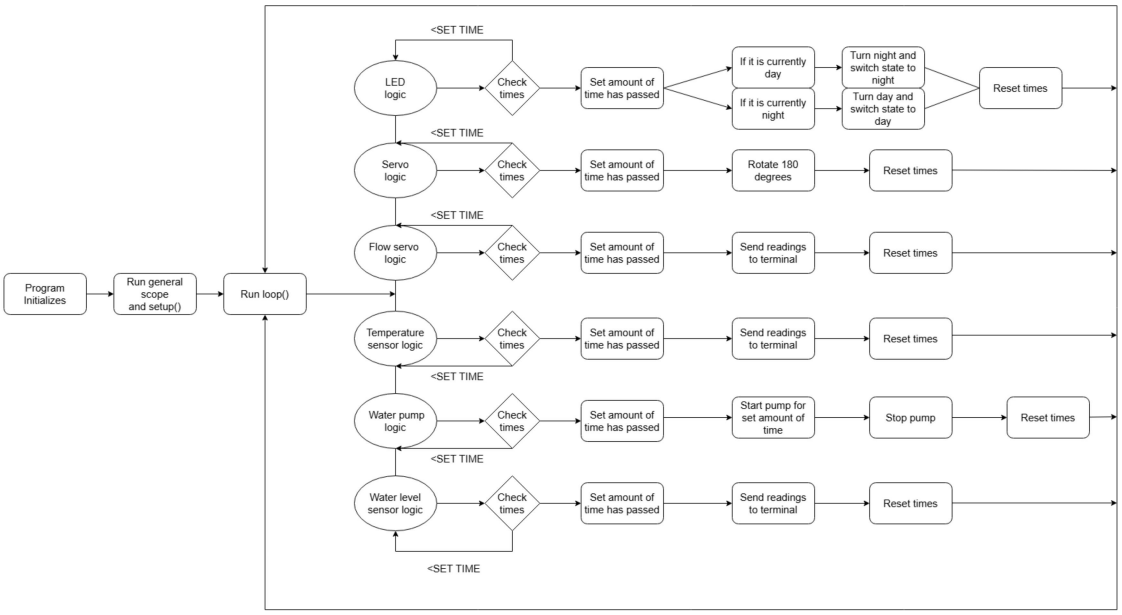



**Figure 28:** Flowchart of the final program

#### 3.2.2.1 Times

For the non-blocking times (3.2.1.1) to work properly, the times need to be set. Different components and different actions within the program are set to run at different times, and to keep track of the different times and easily change each one, several different time constants and variables need to be defined in the general scope of the program. There are other time variables in the different environment's settings (3.22).

```
1    // Times
2    unsigned long lastFeedTime = 0; // Variable to "reset" the time
         after feeding time
3
4    unsigned long lastFlowTime = 0; // Variable to "reset" the time
         after water flow reading
5    const unsigned long flowInterval = 1000; // 1 second interval
         between readings
6
7    unsigned long lastTempTime = 0; // Variable to "reset" the time
         after temperature reading
8    const unsigned long tempInterval = 1000; // 1 second interval
```

```
    between readings
9
10  unsigned long lastLevelTime = 0; // Variable to "reset" the time
        after water level reading
11  const unsigned long levelInterval = 1000; // 1 second interval
        between readings
```

**Listing 3.9:** Time variables set in the general scope

Line 2 in the previous code snippet (3.9) creates a variable that will serve to verify whether the set feeding time has been reached. Lines 4, 7, and 10 have the same task, but verify whether the 1-second interval set in lines 5, 8, and 11 has been reached for an action to take place. Line 13 verifies whether the 10-minute interval between pumps (line 14) is reached. Line 15 is the amount of time the pump remains active once turned on.

Throughout the code, there are several other lines associated with times and where times are utilized.

```
1   //...
2   lastSwitchTime = millis(); // "Reset" day/night cycle timer at
        startup
3   lastFeedTime = millis(); // "Reset" feeder timer at startup
4   //...
5   void loop() {
6       unsigned long currentMillis = millis(); // // currentMillis
            is assigned the current running time
7       //...
8       static unsigned long feedStopTime = 0; // Variable to reset
            the servo after feeding
9       //...
10      if (!feeding && currentMillis - lastFeedTime >= settings->
            feedInterval) { // Non-blocking times
11      lastFeedTime = currentMillis; // "Reset" times for next loop'
            s verifications
12      servo.write(100); // Set speed to 100
13      feeding = true; // Change the servo state
14      feedStopTime = currentMillis; // Starts the 1 second count
            until the servo stops
15      }
16      //...
```

**Listing 3.10:** Times, millis, and non-blocking times example

Lines 2 and 3 in the previous code snippet (3.10) are a reset for the time variables when the program first starts running to make up for any slight runtime before the variables are used in the code. Inside the loop function, in line 6, currentMillis is assigned the current runtime, in milliseconds, at that point. This currentMillis variable will be used to verify every single non-blocking time during the loop. Lines 10 and 17 are examples of the non-blocking times, where, in the first if statement, if the interval is met, the servo will start spinning, the feeding state is set to true (line 8), and feedStopTime is now assigned the current runtime to be used in the next if statement's time verification.

#### 3.2.2.2 Setup function

Code snippet 3.11 is the setup code for the Arduino program. The baud rate is set to 9600 bits per second (line 3), the default starting environment is set to the betta environment (line 6), the feeder servo is attached to pin 8 (line 9) and initialized at an angle of 10 degrees (line 10), the flow sensor's pin is set as an input pin (line 13), the flow sensor pin is set to call the interrupt function (14), the flow sensor's communication is initialized (line 17), the relay pin is set as an output pin (line 20) and initialized turned off (line 21), the LED strip's pin is set as an output pin (line 24) and the time variable used to verify the LED time is reset, removing the initial runtime of the code (line 25), the water level sensor's pin is set as an input pin (line 28), the pump's pin is set as an output (line 31), and, lastly, the pump is initialized turned off (line 32).

```
1   void setup() {
2
3       Serial.begin(9600); // Data rate set to 9600 bits per second
4
5       // Set default environment
6       setEnvironment(BETTA); // Default initial environment when
            the program starts
7
8       // Servo
9       servo.attach(8); // Attach the servo to pin 8
10      servo.write(10); // 10 degrees start position
11
12      // Flow sensor
13      pinMode(flowPin, INPUT_PULLUP); // Set the flow sensor's pin
            as an input pin
14      attachInterrupt(digitalPinToInterrupt(flowPin), Flow, RISING)
            ; // Sets interrupt so that every pulse on flowPin calls
            the Flow function
15
16      // Temperature sensor
17      sensors.begin(); // Initalizes communication with the sensor
18
```

```
19          // Relay
20          pinMode(RELAY_PIN, OUTPUT); // Set the relay pin as an output
                pin
21          digitalWrite(RELAY_PIN, HIGH); // Initialize with the relay
                OFF
22
23          // LED strip
24          pinMode(ledPin, OUTPUT); // Set the LED pin as an output pin
25          lastSwitchTime = millis(); // Define the variable as the time
                that has passed since the program was initialized
26
27          // Water level sensor
28          pinMode(sensorPin, INPUT);  // Set the water level sensor pin
                as an input pin
29
30          // Water pump
31          pinMode(pumpPin, OUTPUT); // Set the water pump pin as an
                output pin
32          digitalWrite(pumpPin, LOW); // Initialize with pump OFF
33      }
```

**Listing 3.11:** Setup function

#### 3.2.2.3 Feeder servo

The feeder servo is programmed to rotate 180 degrees every 24 hours.

Code snippet 3.12 is the feeder servo logic inside the loop function. Line 2 is the variable that controls when the servo moves back to the starting position after spinning to the feeding state angle; line 3 is the Boolean variable that will assign a feeding or non-feeding state to the servo to verify each if statement in lines 9 and 16; line 5 is the if statement containing the jitter control, which makes sure that the servo remains still while not feeding, by setting the speed to the stopping speed in line 6; line 10 attaches the current time in milliseconds to the lastFeedTime variable, recording the last time the servo moved to the feeding angle, to use in the next loop's verifications, until the if statement condition to feed is met; line 11 is the speed of rotation of the servo when feeding; line 12 is the change made to the state of the servo, which in this case is turned to true; this change is important for the if statement on line 16, which stops the servo after feeding; line 13 attaches the current time to feedStopTime, essentially marking the start of the 1-second counter for the servo to move back; line 17 is the stopping speed, post-feeding; and line 18 is the servo feeding state being changed to false, and it will remain in this state until the first if statement's condition is met in future loops.

```
1     // Feeder servo
2     static unsigned long feedStopTime = 0; // Variable to stop the
          servo after feeding
3     static bool feeding = false; // Variable to verify the servo
          state and if it should change
4
5     if (!feeding) {
6         servo.write(90); // Prevent servo from moving
7     }
8
9     if (!feeding && currentMillis - lastFeedTime >= settings->
          feedInterval) { // Non-blocking times
10        lastFeedTime = currentMillis; // "Reset" times for next loop'
              s verifications
11        servo.write(100); // Set speed to 100
12        feeding = true; // Change the servo state
13        feedStopTime = currentMillis; // Starts the 1 second count
              until the servo stops
14    }
15
16    if (feeding && currentMillis - feedStopTime >= 500) { // Non-
          blocking times
17        servo.write(90); // Stop servo after feeding (speed 90)
18        feeding = false; // Change the servo state
19    }
```

**Listing 3.12:** Feeder servo logic

#### 3.2.2.4 Flow sensor

The flow sensor is programmed to output the flow rate to the terminal every second.

Code snippet 3.13 is the flow sensor logic inside the loop function. Line 2 is the if statement with the non-block time for the sensor's output; line 3 attaches the current time to the LastFlowTime variable, recording the last time the flow rate was taken and resetting the time counter; line 5 cancels the interrupt function from working temporarily, and line 6 takes the counts or pulses from the interrupt function (3.14) until that moment and uses them on line 10 to calculate the flow rate in liters per minute, line 7 resets the counter to 0 and line 8 calls and resumes the interrupt function to count the pulses again for the next loop, lines 12, 13, and 14 print out the flow rate to the terminal.

```
1     // Flow sensor
2     if (currentMillis - lastFlowTime >= flowInterval) { // Non-
          blocking times
```

```
3         lastFlowTime = currentMillis; // "Reset" times for next loop'
              s verifications
4
5         noInterrupts(); // Disable interrupts (Flow() will stop)
6         unsigned int pulses = count; // Take pulse counter
7         count = 0; // Reset pulse counter
8         interrupts(); // Re-enable interrupts
9
10        flowRate = pulses / 5.5;  // Turn the pulse count to flow
              rate in L/min
11
12
13        Serial.print("Flow rate: "); // Print result
14        Serial.print(flowRate, 2);    // to the
15        Serial.println(" L/min");     // terminal
16    }
```

**Listing 3.13:** Flow sensor logic

Code snippet 3.14 is the **interrupt function** that is called by every pulse on the flow sensor pin; it is set to increase the count every time it is called. This count is then used in the loop function on the flow sensor logic (3.13) to calculate the flow rate.

```
1     // Flow interrupt
2     void Flow() {  // Interrupt occurs
3         count++;      // Increase count by 1
4     }
```

**Listing 3.14:** Flow interrupt function

#### 3.2.2.5 Temperature sensor

The temperature sensor is programmed to output the temperature to the terminal every second.

Code snippet 3.15 is the temperature sensor logic inside the loop function. Line 2 is the if statement containing the non-blocking times for the sensor's output; line 3 attaches the current time to the LastTempTime variable, recording the last time the temperature was taken and resetting the time counter; line 5 requests the temperature reading from the sensor; and line 6 gets the temperature in Celsius and attaches it to the temperature variable. Lines 8, 9, and 10 print out the temperature to the terminal.

```
1     // Temperature sensor
2     if (currentMillis - lastTempTime >= tempInterval) { // Non-
          blocking times
3         lastTempTime = currentMillis; // "Reset" times for next loop'
              s verifications
4
5         sensors.requestTemperatures(); // Get temperature measurement
              at this moment
6         temperature = sensors.getTempCByIndex(0); // Get temperature
              at Celsius
7
8         Serial.print("Temperature: "); // Print
9         Serial.print(temperature);     // to the
10        Serial.println(" C");          // terminal
11    }
```

**Listing 3.15:** Temperature sensor logic

#### 3.2.2.6 Relay-heater relation

The relay is programmed to turn the heater on or off whenever a certain temperature is reached.

Code snippet 3.16 is the relay logic inside the loop function, which controls the heater state, turning it on or off. Line 2 is the Boolean variable that will assign an on or off state to the relay and, consequently, the heater. Lines 4 and 8 are the if statements containing the non-blocking times. When the heater is off, and the temperature reaches a certain minimum value, the relay will close the circuit (line 5), causing the heater to turn ON (line 6), and when the heater is on, and the temperature reaches a certain maximum value, the relay will open again (line 9), making the heater turn off (line 10)

```
1     // Relay (heater)
2     static bool heaterOn = false; // Set relay-heater state
3
4     if (!heaterOn && temperature <= settings->minTemp) { // Check
          state and temperature
5         digitalWrite(RELAY_PIN, LOW); // Relay ON
6         heaterOn = true; // Set relay-heater state as ON for next
              loop's verification
7     }
8     else if (heaterOn && temperature >= settings->maxTemp) { // Check
          state and temperature
9         digitalWrite(RELAY_PIN, HIGH); // Relay OFF
```

```
10          heaterOn = false; // Set relay-heater state as OFF for next
                loop's verifications
11      }
```

**Listing 3.16:** Relay-heater relation logic

#### 3.2.2.7 LED strip

The LED strip is programmed to turn on or off after a certain amount of time has passed.

Code snippet 3.17 is the LED strip logic used in the day/night cycle for the aquarium, inside the loop function. Lines 2 and 7 are the if statements containing the non-blocking times; line 3 is the Boolean variable that will assign a day/night, on/off state to the LED strip. In this case, it sets the state to daytime. This variable is used to verify the non-blocking times. If it is currently daytime and a certain set amount of time has passed, the state will change to night; similarly, if it is currently nighttime and another certain amount of time has passed, the state will, this time, change to day. Lines 4 and 9 attach the current runtime of the program to the LastSwitchTime variable, marking the time of the last switch from the day/night states. Line 5 sets the brightness to the daytime set brightness, line 8 changes the Boolean variable to true (day), and line 10 changes the brightness to the set nighttime brightness.

```
1       // LED strip
2       if (isDay && currentMillis - lastSwitchTime >= settings->
            dayDuration) { // Check state and times
3           isDay = false; // Set to night state
4           lastSwitchTime = currentMillis; // "Reset" times for next
                loop's verification
5           analogWrite(ledPin, settings->nightBrightness); // Set LED
                strip to set night brightness
6       }
7       else if (!isDay && currentMillis - lastSwitchTime >= settings->
            nightDuration) { // Check state and times
8           isDay = true; // Set to day state
9           lastSwitchTime = currentMillis; // "Reset" times for next
                loop's verification
10          analogWrite(ledPin, settings->dayBrightness); // Set LED
                strip to set day brightness
11      }
```

**Listing 3.17:** LED strip logic for day/night cycle

#### 3.2.2.8 Water level sensor

The water level sensor is programmed to output whether there is water present or not.

Code snippet 3.18 is the water level sensor logic inside the loop function. Line 2 is the variable that will be used to verify whether there is any liquid detected by the water level sensor. Line 4 is the if statement containing the non-blocking time for the sensor's output; line 5 attaches the current runtime of the program to LastLevelTime, marking the moment the sensor last outputted information. Lines 7 and 9 are if-else statements used to output the readings of the water level sensor. If the sensor outputs low voltage (or no voltage), that means that water has been detected by the sensor; on the other hand, if the sensor outputs high voltage, there is no water present. The terminal messages for both cases are handled by lines 8 and 10.

```
1     // Water level sensor
2     int sensorState = digitalRead(sensorPin); // Check pin state (
          HIGH or LOW)
3
4     if(currentMillis - lastLevelTime >= levelInterval){ // Non-
          blocking times
5         lastLevelTime = currentMillis; // "Reset" times for next loop
              's verification
6
7     if (sensorState == LOW) { // Liquid detected
8         Serial.println("Status: [ LIQUID DETECTED ]"); // Terminal
              message
9     } else { // No liquid detected
10        Serial.println("Status: No liquid..."); // Terminal message
11    }
12    }
```

**Listing 3.18:** Water level sensor logic

#### 3.2.2.9 Water pump

The water pump is programmed to pump water, continuously, for 5 minutes, every 5 minutes.

Code snippet 3.19 is the water pump logic inside the loop function. Line 2 is the Boolean variable that represents the current state of the pump to help verify the times in lines 10 and 16. Line 3 is the variable that will store the time of the last second, so that line 5 can properly work every second. Line 6 attaches the current runtime of the program to OneSecondAgo to help the next loop's if statements' verifications. Line 7 increases the

pumpCount by 1 whenever a second passes, serving as a timer that will be used to verify whether or not 5 minutes have passed (300 seconds) in lines 10 and 16. Lines 11 and 17 control the pump, either on/HIGH or off/LOW, whenever the state should change (lines 12 and 18). Lines 13 and 19 reset the time counter for the future loop.

```
1     // Water pump
2     static bool pumpOn = false; // Set pump state
3     static unsigned long oneSecondAgo = 0;  // Variable to help count
          the seconds
4
5     if (currentMillis - oneSecondAgo >= 1000) { // 1 second non-
          blocking time
6       oneSecondAgo = currentMillis; // Attach the current time to
            oneSecondAgo
7       pumpCount++; // Seconds time counter
8     }
9
10    if (!pumpOn && pumpCount >= 300) { // Check if 5 minutes have
          passed
11      digitalWrite(pumpPin, HIGH); // Start pump
12      pumpOn = true; // Change pump state
13      pumpCount = 0;  // Reset pumpCount for next loops
14    }
15
16    if (pumpOn && pumpCount >= 300) { // Check if 5 minutes have
          passed
17      digitalWrite(pumpPin, LOW); // Stop pump
18      pumpOn = false; // Change pump state
19      pumpCount = 0;  // Reset pumpCount for next loops
20    }
```

**Listing 3.19:** Water pump logic

#### 3.2.2.10 Environments

Different environments are present in the program; currently, there are two: the guppy fish and the betta fish environments, both defined in the global scope of the program. They hold different settings, times, and conditions (18) and can be switched at any time during the running of the program. The setup of the environments begins with creating the **environment constants** BETTA and GUPPY (3.20 in lines 3 and 4) and starting the program with the BETTA environment (line 7).

```
1     // Different environments setup
2     enum Environment {
```

```
3     BETTA,
4     GUPPY,
5     // More environments can be added later
6     };
7     Environment currentEnv = BETTA;  // Default environment
```

**Listing 3.20:** Environment constants enumeration

The next step, still in the global scope, is to create all the **settings/variables** that can alternate between different environments, variables that represent temperature ranges, day/night cycle duration, and feeding schedules. Line 1 creates a group of variables inside the AquariumSettings structure; lines 2 and 3 are variables that control the temperature range inside the aquarium; lines 4 and 5 are the duration of the day and night cycles; line 6 is the feeding schedule interval; lines 7 and 8 control the brightness of the LED strip for the day and night cycles; and line 11 creates a pointer for the AquariumSettings that cannot change the variables inside the structure.

```
1     struct AquariumSettings {
2         float minTemp;
3         float maxTemp;
4         unsigned long dayDuration;
5         unsigned long nightDuration;
6         unsigned long feedInterval;
7         int dayBrightness;
8         int nightBrightness;
9     };
10
11    const AquariumSettings* settings;
```

**Listing 3.21:** Environment settings/variables

The remainder of the code for the environment, set in the global scope, is the actual variables for each one of the environments. Lines 2 and 9 set the minimum and maximum values for the temperature in both environments; lines 3 and 10 set the duration of the day and night cycles; lines 4 and 11 set the feeding interval or schedule (equal for both); and lines 5 and 12 set the brightness of the LED strip at day and night times (equal for both).

```
1     const AquariumSettings betta = {
2         27.5, 28.5,                  // Temperature range (C) (around
              28)
3         50400000UL, 36000000UL,       // 14h day / 10h night
4         86400000UL,                  // Feed every 24h
```

```
5         255, 0                          // Light ON / OFF
6     };
7
8     const AquariumSettings guppy = {
9         26.0, 27.0,                   // Temperature range (C) (around
              26.5)
10        43200000UL, 43200000UL,        // 12h day / 12h night
11        86400000UL,                   // Feed every 24h
12        255, 0                         // Light ON / OFF
13    };
```

**Listing 3.22:** Betta and Guppy environment conditions

The setEnvironment function (3.23) changes the aquarium's operating mode when called. Line 2 stores the current environment and saves it; line 4 is the switch statement that checks the current environment (in lines 5 and 10) and changes the settings accordingly (in lines 6 and 11) by using the settings pointer. Whenever there is an environment switch, a message will be printed out to the terminal (lines 7 and 12); lines 8 and 13 exit the switch so that the switch does not continue into the next case. Line 16 ensures the program initializes in the daytime state, and line 17 sets the LED strip to the daytime brightness. Lines 19 and 20 attach the current time to the last time there was a switch in the environment and the last time the servo changed its state to feeding and rotated to the associated angle.

```
1     void setEnvironment(Environment env) {
2         currentEnv = env;
3
4         switch (env) {
5             case BETTA: // Check if current environment is betta
6             settings = &betta; // Settings points to betta
                  configuration
7             Serial.println("Environment: Betta"); // Terminal message
                   when switching environment
8             break; // Exit after executing branch
9
10        case GUPPY: // Check if current environment is guppy
11            settings = &guppy; // Settings points to guppy
                  configuration
12            Serial.println("Environment: Guppy"); // Terminal message
                   when switching environment
13            break; // Exit after executing branch
14            }
15
16        isDay = true; // Initialize environment in daytime state
17        analogWrite(ledPin, settings->dayBrightness); // Set LED
              strip to daytime setting
```

```
18
19          lastSwitchTime = millis(); // "Reset" day/night cycle timer
20          lastFeedTime = millis(); // "Reset" feeder timer
21      }
```

**Listing 3.23:** Function to change aquarium environments

Code snippet 3.24 sets the environment switching conditions. Line 2 detects whether there is an input on the terminal; line 3 creates a character variable, cmd, that will be attached the serial input character; line 5 is the switch statement, which will read the cmd character, and if the character is b or B (lines 6 and 7), the environment will change to BETTA. If the character is g or G (lines 11 and 12), the environment will change to GUPPY. Lines 9, 14, and 18 prevent the code from running further and possibly changing the environment twice. If the character is none of the aforementioned, a message will be displayed in the terminal saying "Unknown command".

```
1       // Environment switching
2       if (Serial.available() > 0) { // Check for terminal input
3           char cmd = Serial.read(); // Attach the terminal input to cmd
                variable
4
5           switch (cmd) { // Read cmd variable
6               case 'g': // If input is character g
7               case 'G': // If input is character G
8                   if (currentEnv == GUPPY) { // Check current
                        environment
9                       Serial.println("Guppy Environment Already Applied
                            "); // Message to terminal
10                  } else {
11                      setEnvironment(GUPPY); // Change environment to
                            GUPPY
12                  }
13                  break; // Exit after executing branch
14
15          case 'b': // If input is character b
16          case 'B': // If input is character B
17              if (currentEnv == BETTA) { // Check current environment
18                  Serial.println("Betta Environment Already Applied");
                        // Message to terminal
19              } else {
20                  setEnvironment(BETTA); // Change environment to BETTA
21              }
22              break; // Exit after executing branch
23
24          default:
```

```
25          Serial.println("Unknown command");
26          break; // Exit after executing branch
27          }
28      }
```

**Listing 3.24:** Environment switching cases

# Chapter 4

# Testing and discussion

## 4.1 Dry testing

With all the components assembled as stated in figure 25, the code was run to verify functionality and to look for errors. For this testing, the heater was not connected to a power source since dry testing, as stated in the product, in the form of a warning, could prove dangerous. The heater should only be used and tested in water.

After running the program, the LED strip light turned on, and all the sensors' readings were displayed properly on the Arduino IDE's terminal. The **temperature sensor** was tested by applying body heat to it; this way, the **relay** was also tested, since when the temperature sensor's reading reaches a certain minimum or maximum set temperature, the relay turns off or on accordingly. When the temperature rose above 28.5°C, the relay turned off. This action by the relay was made clear and signaled by a ticking sound and by the change of an LED embedded in the component itself. After waiting for the sensor to cool down to the minimum set temperature, the relay turned back on. The **water flow sensor** was tested by blowing air into it and checking the terminal for changes. Variations were verified, and so the sensor was working properly. To check for the **LED strip**'s change when switching between day and night states, the relevant times in the code were changed to a couple of seconds, instead of the several hours it would take to change cycles. Once the LED strip proved to be working, the **servo** was tested. It was tested in the same way as the LED strip was; the feeding schedule time was changed in the code to a couple of seconds, and the servo moved as it should. Finally, the **environment switching** was tested by entering the characters 'g', 'G', 'b', and 'B' into the IDE's terminal. The environment switching should reset all the times and set the LED state to daytime, but occasionally, the LED would turn off when switching environments. This might have been caused by an error when switching to an already set environment, for example, "switching" from the BETTA environment to BETTA. The fixed code block is in snippet 3.24.

## 4.2 Testing with water

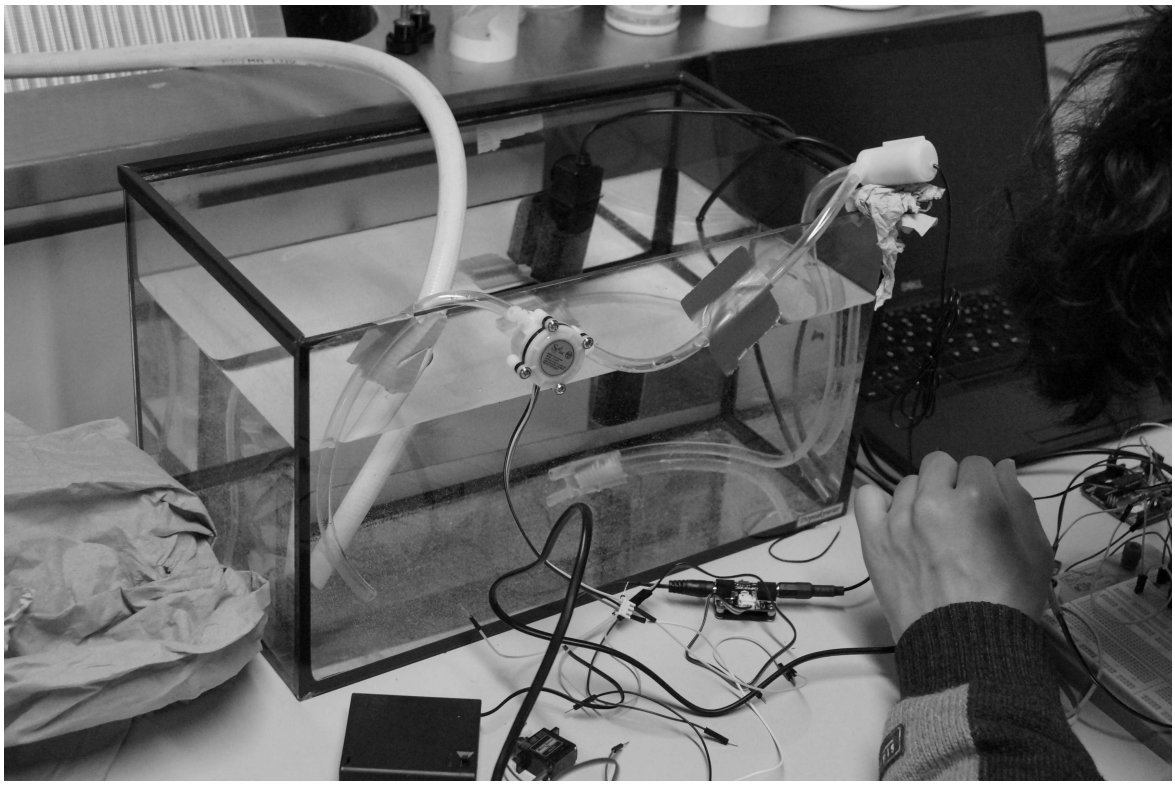


**Figure 29:** Final assembly, for testing

**Servo issue:** When the time came to test the prototype's functionality in water, the SG90 micro servo, used until then, had its wires accidentally removed from its internals. The SG90 micro servo is positional, meaning it is controlled by changing the angle of rotation in the code. This micro servo was replaced by another of the same name but not of the positional type; the new servo is continuous, meaning it is controlled by the speed of rotation. A continuous servo poses an inconvenience at the time of testing, since it is required to know the speed and the amount of time it takes for the servo to rotate to a certain degree.

And so, for this reason, tests were run, and a conclusion was reached. After testing the directional and continuous servo for several different times and speeds, the most optimal was 1220 ms (4.1, line 13) at 100 speed (4.1, line 12); this made the servo spin 360 degrees, once, at a slow pace, which allowed the food from the feeder to drop down into the aquarium and the servo to finish its rotation. The servo spun several times; there was a small deviation from the starting position, but after 2 to 4 spins, it returned to the default

position. Line 15 (4.1) stops the servo at whatever angle it is for 2 seconds (4.1, line 15).

```
1   #include <Servo.h> // Servo library
2
3   Servo servo; //
4
5   void setup() {
6     servo.attach(8);
7     servo.write(90);
8     delay(2000);
9   }
10
11  void loop() {
12    servo.write(100);
13    delay(1220);
14    servo.write(90);
15    delay(2000);
16  }
```

**Listing 4.1:** Test code for the SG90 continuous servo

It was later noticed that the servo deviated more than usual at some point; this was due to the load applied to it. Since the servo cannot be controlled by the angle of rotation, only time controls how far the servo spins; this movement's efficiency and time are affected by the load the servo needs to carry when rotating. This is something to be taken into consideration when the prototype is fully assembled and the feeder cylinder is attached to the servo.

When it came to integrating the servo's code into the Arduino program, the deviation seen before was augmented; not only that, it was severely irregular. This may be due to the fact that the final code uses non-blocking times and not delay(). No change in the previously assessed optimal time for the rotation made the servo spin correctly; it was always irregular. The last time tested was 500 ms. This last attempt resulted in the servo rotating 180 degrees with little to no deviation, and similarly to the simple test code 4.1, whenever there was a change in the angle of rotation in any of the spins, it would simply fix itself in 2 to 4 cycles. This led to a change in the design of the feeder cylinder, which will now be two-sided (two openings). Now, instead of the servo spinning 360 degrees once every 24 hours, it will do a single 180-degree spin every 24 hours.

The old code block for the feeder servo is code snippet 4.2. The new servo logic is code snippet 18, in the software chapter.

```
    // Feeder servo
    static unsigned long feedMoveBackTime = 0; // Variable to reset
        the servo after feeding
    static bool feeding = false; // Variable to verify the servo
        state and if it should change

    if (!feeding && currentMillis - lastFeedTime >= settings->
        feedInterval) { // Non-blocking times
        lastFeedTime = currentMillis; // "Reset" times for next loop'
            s verifications
        servo.write(180); // Change the angle to 180 degrees
        feeding = true; // Change the servo state
        feedMoveBackTime = currentMillis; // Starts the 1 second
            count until the servo resets position
    }

    if (feeding && currentMillis - feedMoveBackTime >= 1000) { // Non
        -blocking times
        servo.write(10); // Reset servo after feeding
        feeding = false; // Change the servo state
    }
```

**Listing 4.2:** Old feeder servo logic

After fixing the servo issue, the testing proceeded without the heater and without the water flow sensor because the heater needs insulation in the ground wire, and since it is a high-voltage component, it could prove dangerous. The water flow sensor cannot be submerged in water without damaging the component; it needs to be inserted in a tube where a water pump will create the flow.

The testing gave good results; the temperature sensor worked as it should, and while hot water was being added to the already existing water in the tank, the temperature sensor output substantially higher values. The relay also reacted to the change in temperature and made a ticking sound, as well as switched off the embedded LED light. The water level sensor kept displaying the "LIQUID DETECTED" message when there was no water near it and when there was no more water being added to the aquarium.

**Water level sensor issue:** Since the water level sensor seemed not to be working properly, it was again tested by itself, using the code in snippet 4.3. The results were inconsistent and unreliable. No change to the code, wiring, or pins on the Arduino seemed to fix the issue.

```
int sensorPin = 7;

void setup() {
  Serial.begin(9600); // Set baud rate
  pinMode(sensorPin, INPUT_PULLUP) // Level sensor as an input
}
void loop() {
  readValue = digitalRead(sensorPin); // Take value from sensor
  Serial.print("Value = ");  // Print to the
  Serial.println(readValue); // terminal
  delay(1000); // 1 second interval
}
```

**Listing 4.3:** Water level sensor test code

**Water pump issue:** When testing the final Arduino program, with the implementation of the water pump, issues arose. The water pump would either work for some time and then stop fully and never start working again. After some changes in the code, the water pump simply worked constantly, without stopping. No changes to the code seemed to get the pump to work as intended. The final, non-working code, is in snippet 4.4.

```

// Water pump
static bool pumpOn = false; // Set pump state
static unsigned long pumpStartTime = 0; // When pump turns ON

if (!pumpOn && currentMillis - lastPumpCycleTime >= pumpInterval)
    {
  pumpStartTime = currentMillis; // Start duration timer
  lastPumpCycleTime = currentMillis;  // Reset interval timer
  digitalWrite(pumpPin, HIGH);
  pumpOn = true;
}
if (pumpOn && currentMillis - pumpStartTime >= pumpDuration) {
  digitalWrite(pumpPin, LOW);
  pumpOn = false;
}
```

**Listing 4.4:** Old water pump logic

### 4.2.1 Plotting graph using MATLAB

The servo's rotation time was changed for the sake of testing and plotting the readings. And so, now the servo was set to rotate 180 degrees every 5 minutes. But when the code

was uploaded to the Arduino, the microservo would twitch/jitter every 5 to 6 seconds. To fix this, a jitter control was added (see 3.12), and the jittering was quickly fixed.

Graph 30 plots the sensors' readings and the actuators' activity throughout the span of 30 minutes (displayed in seconds), in real time, using MATLAB. This small 30-minute sample size is a way to examine how the setup would behave throughout a larger time span. Every minute, the water pump started working for 1 minute's duration, which created a water flow sensed by the flow sensor. The feeder's servo was activated every 5 minutes. The heater relay remained on for the entirety of the sampling time, and the temperature kept steadily going up, as intended. At the time of taking the samples and plotting the graph, the active environment was BETTA, with a temperature around 27°C. If the heater did not propose a limitation to the temperature control, the temperature would, most likely, settle anywhere between the intended temperature levels, as shown in figure 31.

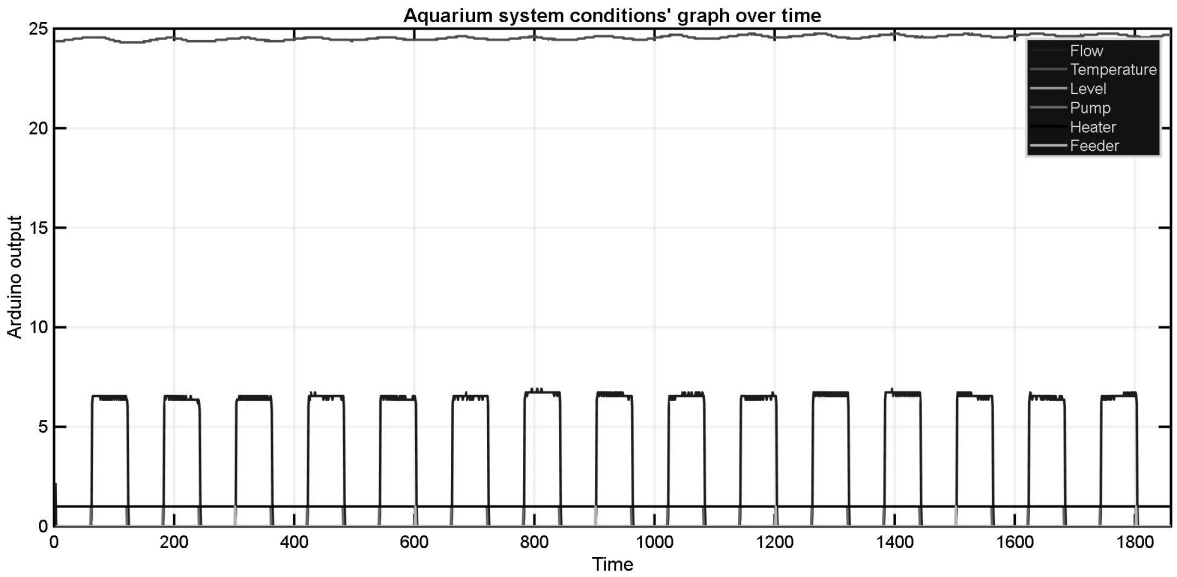



**Figure 30:** 30-minute sample graph, in seconds

Graph 31 is a prediction of what the graph would look like, made by 365 Copilot AI. It is very similar to the real-time graph plotted by MATLAB when testing. The major difference between the two graphs is the heater relay and the temperature, since, as explained previously, the heater presents a major limitation in controlling the temperature of the water. Everything else worked exactly as predicted by the AI model.

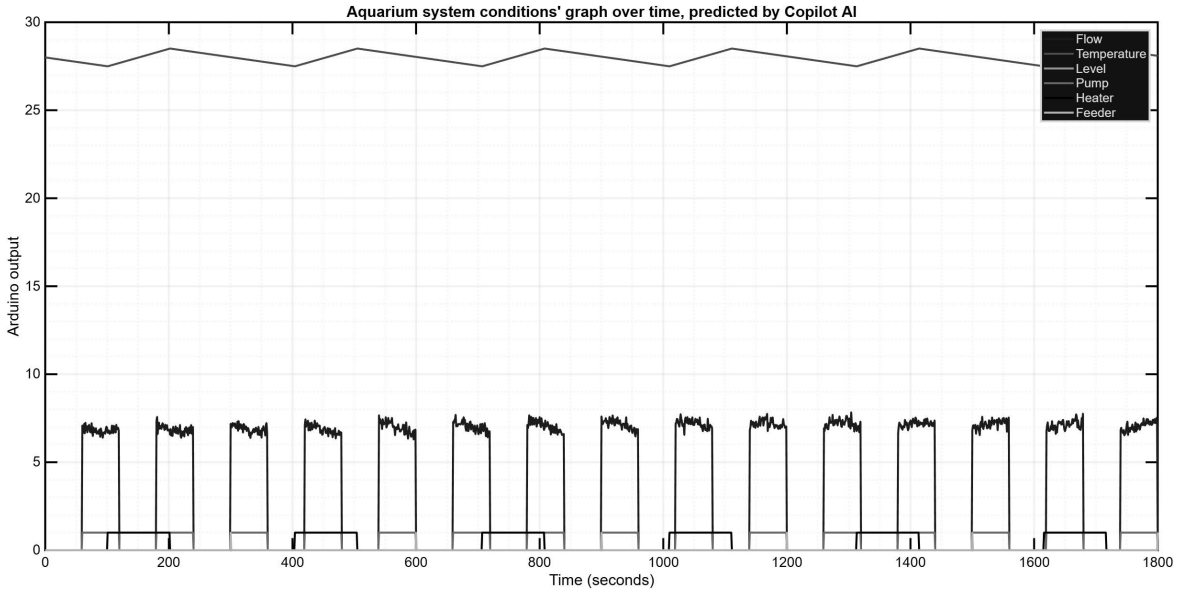



**Figure 31:** 30-minute sample graph prediction by Copilot AI

To more clearly visualize the results, and with more detail, graph 32 displays the same plotted conditions as the previous graph (30), but with 3.5 minutes of sampling time. At second 600, the feeder servo activated (cyan line) for a brief moment, rotating 180 degrees, as programmed.

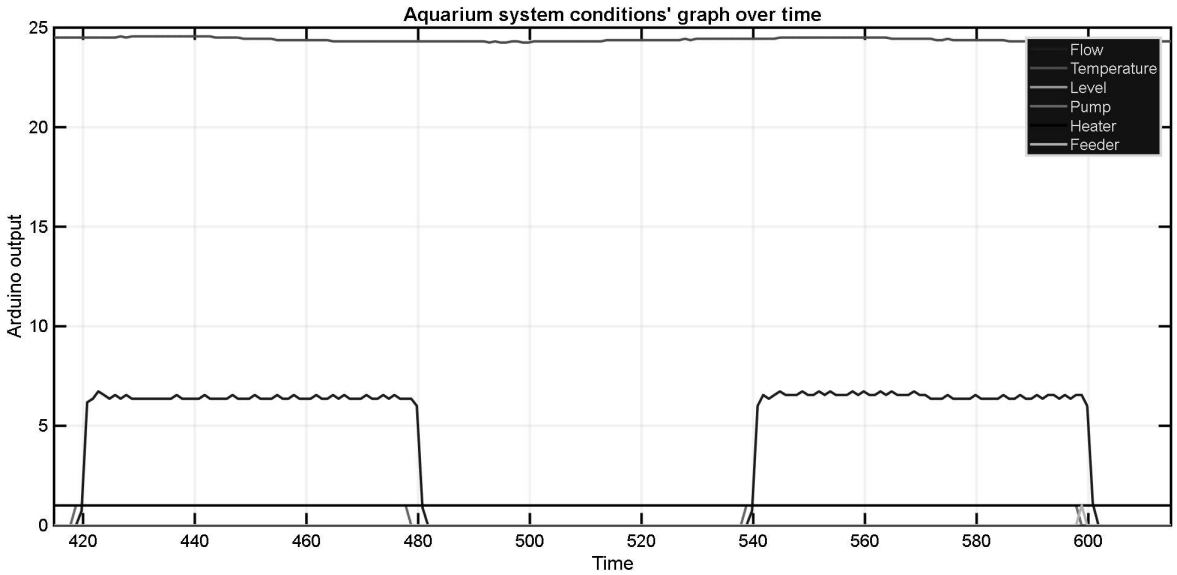



**Figure 32:** 3.5-minute sample graph, in seconds

The prototype was tested while being powered on by a regular power bank inserted in the USB port of the Arduino. The power bank had a capacity of 4400 mAh and a DC output of 5 V and 1 A. The system seemed to be working as intended when using the power bank as the power supply.

### 4.2.2 Discussion

The purpose of the project is to investigate how an automated system could be designed and evaluated to support stable conditions in a domestic aquarium through automated feeding, temperature control, lighting, and water flow monitoring while remaining low-cost. The testing results demonstrate that an Arduino-based prototype can reliably automate several essential aquarium maintenance/management tasks mentioned above, although with a few issues present due to budget and hardware limitations.

The project demonstrated that automating stable temperature, feeding schedules, lighting cycles, and water circulation within an aquarium is complex and requires knowledge of microprocessor architecture, sensor/actuator logic control, and a basic understanding of embedded systems. Various small-scale digital components were connected to the Arduino microcontroller, such as a temperature sensor and a servo motor, etc. These components were programmed in C++ using the Arduino IDE with various libraries accompanying the parts. The control logic was designed so that most functions could run automatically. The use of timers and non-blocking logic allowed the system to perform multiple tasks at the same time, such as monitoring sensors while controlling outputs independently. In the testing, this method of programming showed that carefully implemented timing and non-block logic are important in a microprocessor-based system, especially when the servo and pump did not behave as expected, as mentioned in 4.2.

The testing results demonstrate that most of the aquarium conditions and parameters could realistically be monitored and controlled using the low-cost hardware. The temperature sensor and relay operated reliably during both dry and water testing, while the water flow sensor and the heater needed certain conditions such as water presence in the tank. The feeder servo also automated feeding schedules, although several adjustments were needed after replacing the original positional SG90 servo with a continuous servo as noted in 4.2. This issue highlighted the importance of testing, calibration, and evaluating the compatibility of chosen hardware components.

However, the testing also showed some limitations of the current aquarium management system. The water level sensor gave inconsistent readings, even when it was tested separately, and the feeder rotations are inconsistent every 4 turns. These problems show that, even though low-cost components can be used to build a functional prototype, their reliability must be thoroughly evaluated. The servo issue also showed that replacing one

component with another similar-looking component can change the whole control method, since a positional servo and a continuous servo behave differently.

Furthermore, the testing process was important for verifying the system. Dry testing made it possible to safely check the circuit, code, and component behavior before using water and high-voltage components. Testing with water gave a more realistic view of how the system behaved in actual aquatic conditions. The MATLAB graphs helped document the real-time sensor readings and actuator activity, which made it easier to evaluate whether the system worked as intended over time. Nevertheless, to fully demonstrate whether the system creates a livable condition, an environment resembling a normal aquarium with plants and fish must be created and tested. The MATLAB graph 30 also demonstrated a very slow but steady increasing temperature as the heater is activated. There were no delays of said coder timers in the program regarding the servo turn and water pump as seen on graph 32. Moreover, the comparison between graphs 31 and 30 provides insight into how similar the results are to the AI prediction of the expected outcome, indicating that the system is working as it should.

Overall, the project demonstrated that a low-cost Arduino-based embedded system can be designed, programmed, documented, and tested to support stable aquarium conditions to a reliable extent, where all the components except one work. The prototype fulfilled most of the design specifications, especially for temperature control, lighting, feeding, and water flow monitoring. At the same time, results showed that some components need further improvement before the system can be considered fully reliable for long-term use. In the process, the group gained practical experience with embedded programming, microcontroller-based system design, sensor and actuator interfaces, testing, troubleshooting, and critical evaluation of the final system. The total cost of the components in table **??** was $106.42USD, with the addition of $42.72USD from the easy relay module, the glass tank and the battery holder which were not mentioned, it totals to $149.14USD. For a 25L smart aquarium, this cost is a steep decrease compared with commercial aquariums such as the Apex Pro, which costs $990 USD, or the Hydros Control X4 PRO pack, which costs around $662 USD, as mentioned in subsection 1.4.4.

# Chapter 5

# Future work

There are certain additions and modifications that can still be made to the prototype, in order to make a more robust solution. These changes could be, to name a few:

- Make a custom PCB or use a sensor shield
- Implement more sensors like turbidity, ammonia level, and dissolved oxygen sensors
- Add filtration system
- Add more different environments
- Button to cycle between the environments
- LCD panel to check conditions and current environment
- Change the heater to remove the temperature limitation
- Remotely control the system
- Make a graphical user interface (GUI) for user-friendly control

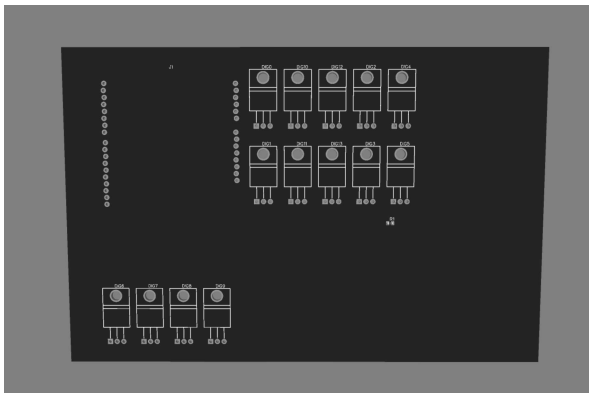


**Figure 33:** PCB front view

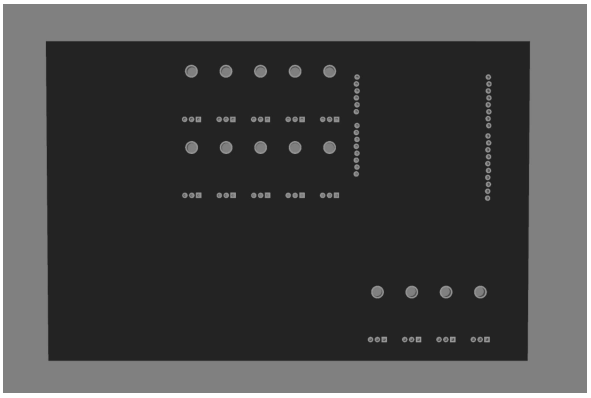


**Figure 34:** PCB back view

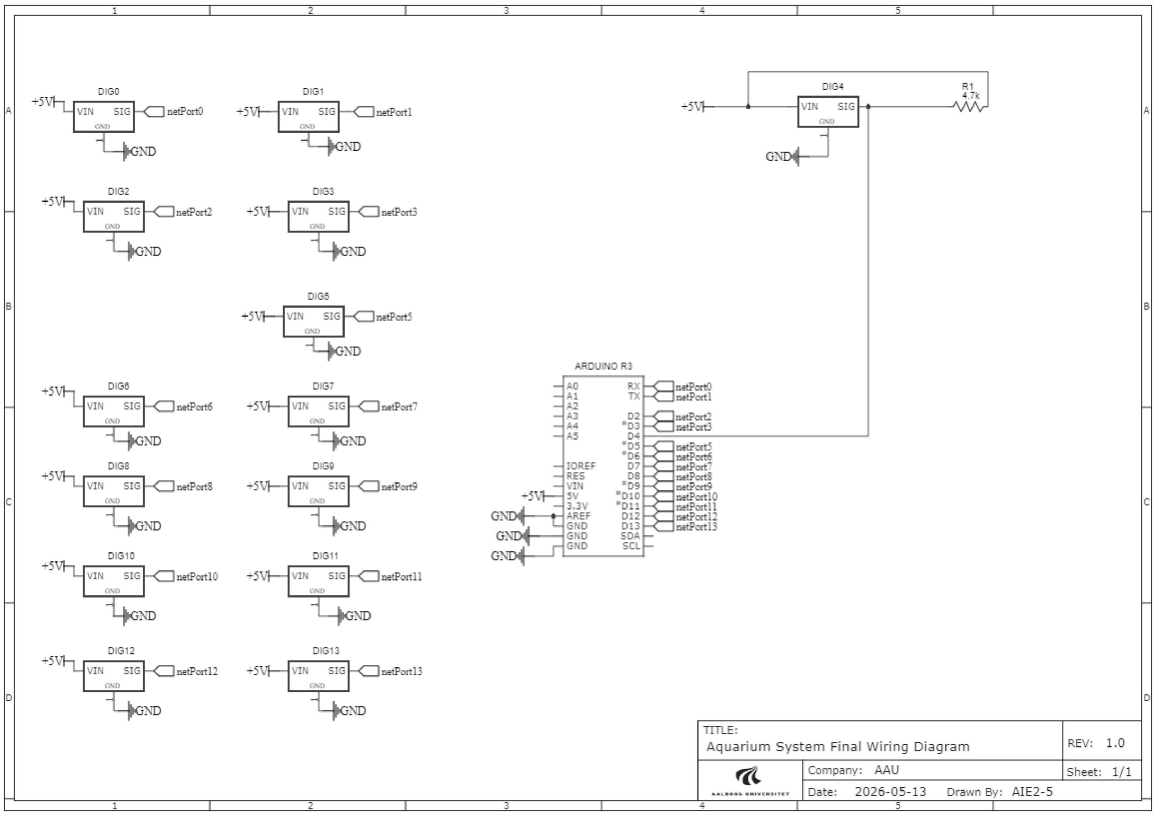



**Figure 35:** Custom PCB diagram

**Custom PCB:** The custom PCB designed for the aquarium only has the connections for the digital pins' setup (see diagram 35) since they are the only ones needed for the sensors' inputs. In the case of the impossibility of properly designing and creating a custom PCB, a sensor shield for the Arduino UNO R3 could be acquired, which would make the assembly cleaner, tidier, and easier, as well as making it more presentable and complete.

**Additional sensors:** To make the system more complete, certain sensors could be added to the full, final assembly. A turbidity sensor could be added to verify the clearness of the water and to check whether or not it should be changed/cleaned. An ammonia level sensor to check the ammonia levels in the water, since this fish waste product can prove to be extremely detrimental to the well-being of the life within the aquarium. Ammonia is continually introduced into your aquarium through fish respiration, and it burns and damages the fish's gills, limiting their ability to absorb oxygen. When reaching certain ammonia levels, a filtration system is activated to clear the water of the dangerous ammonia. A dissolved oxygen sensor could also be added to activate the water pump whenever the oxygen levels in the water reach a certain low value, since the movement helps oxygenate the water.

**Environments:** In order to make the system more capable and to add depth to the program, more environments could be added. Environments with more divergent conditions than the "Betta" and "Guppy" environments, since both of these are somewhat similar. A button should also be added to cycle through the environments until the desired one is reached. To visualize the environment selected as well as the current conditions of the aquarium, without the need for the Arduino IDE's terminal, an LCD panel should be implemented.

**Heater:** The heater used in the final prototype of the system presents certain limitations due to the 24-26°C range (see 2.1.7); therefore, a different water heater, with a wider temperature range, fits the needs of the design better and would add more versatility when it comes to keeping different types of environments.

**GUI:** The existence of an interface allows for a more user-friendly experience for the system. It would still require an external device, like a laptop or cell phone, in order to monitor and control the conditions and switch environments without the need for the Arduino IDE terminal. Figure 36 is an example of an interactive, simple interface.

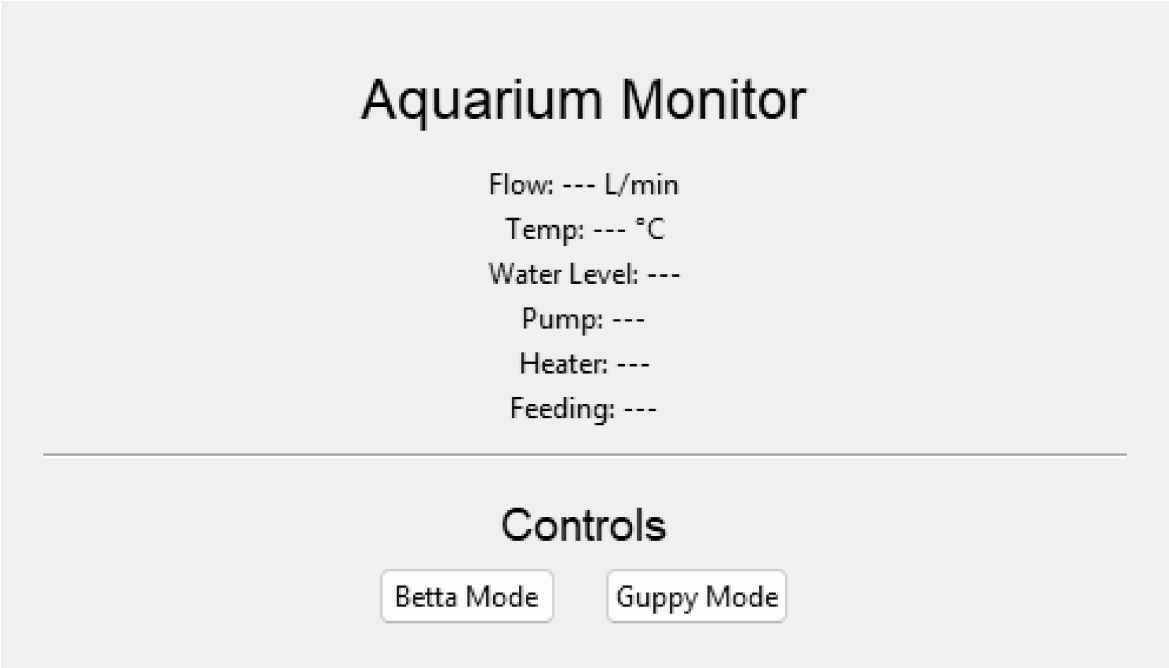



**Figure 36:** GUI example

# Chapter 6

# Conclusion

The aim of this report and project was to investigate whether automation can reduce human error and support stable conditions within a domestic aquarium. The prototype was designed to automate an aquarium's key functions, such as feeding, lighting, temperature, water flow, and water level. These functions are important, as changes in water conditions can negatively affect the fauna and flora within the aquarium.

To address our problem, an Arduino-based prototype was designed and built. Based on the current prototype design, the project statement was partially fulfilled. The Arduino serves as the prototype's central control unit, reading sensor inputs, processing them, and controlling multiple actuators based on that information. The current system consists of a temperature sensor, a relay module, a microservo, an LED strip, a water level sensor, a heater, a water flow sensor, and a water pump. With this design, it is possible to reduce human error and rely on automation to respond to changes in water conditions and feed the fish.

Although a low-cost Arduino-based system was successfully developed and can automate feeding, lighting, water flow monitoring, and basic temperature regulation, there are a few problems still to be addressed in the prototype's design and development. While the heater does work, its main limitation is that it can't reach a temperature above 26°C. The temperature limitation doesn't directly contradict the problem statement, but the prototype was designed to support two specific environments, one of which requires 28°C; for this reason, its considered a limitation. Additionally, it was originally intended to include a water level sensor, but during testing, it was deemed faulty and wasn't included in the prototype. This meant that not all water conditions could be evaluated.

With the final cost of the prototype being $149.14 USD, the project demonstrates that a low-cost Arduino-based system can automate several aquarium maintenance tasks; the end results also highlight how cheaper components may not be as reliable as other suitable

options. Future improvements could be made primarily by replacing the heater with a more suitable model and, if necessary, upgrading or replacing the water level sensor. Additional sensors could be added to regulate a wider range of conditions, such as pH, dissolved oxygen, and salinity.

In conclusion, the problem statement was partially addressed. The system was low-cost and successfully regulated feeding, lighting, water flow, and temperature. Despite the limitations of the heater and water level, the prototype demonstrated that automating aquarium management can be affordable and practical, with a few improvements needed before it could be used reliably in a domestic aquarium.

# Bibliography


HYDROS Control X4 Starter Pack. Nov. 2025, Accessed: 20 May 2026. www.theaquaponicsource.com/shop/backup-monitoring-systems/hydros-control-x4-starter-pack/.

IUCN/SSC Freshwater Fish Specialist Group. About Freshwater Fishes. 2026, Accessed 22 Mar. 2026.

Lichak, Madison R, et al. "Care and use of Siamese fighting fish (Betta splendens) for research". *Comparative medicine*, vol. 72, no. 3, 2022, pp. 169–80.

Neptune Systems. A3 Apex Pro Aquarium Controller System. Accessed: 20 May 2026. www.bulkreefsupply.com/apex-a3-pro-controller-system-neptune-systems.html.

Neptune Systems. Apex A3 Series. Sept. 2025, Accessed: 20 May 2026. www.neptunesystems.com/apex-a3-series/.

Periyadi, Periyadi, et al. "IoT-based guppy fish farming monitoring and controlling system". *TELKOMNIKA (Telecommunication Computing Electronics and Control)*, vol. 18, no. 3, 2020, pp. 1538–45.

Pethiyagoda, PDRS, et al. "Salinity tolerance of wild Poecilia reticulata (guppy) under laboratory conditions". 2019.

Reid, G McG, et al. "Global challenges in freshwater-fish conservation related to public aquariums and the aquarium industry". *International Zoo Yearbook*, vol. 47, no. 1, 2013, pp. 6–45.

Shah, Tarang Kumar, et al. "Effect of temperature on growth and survival of guppy (Poecilia reticulata)". *J. Exp. Zool. India*, vol. 20, no. 1, 2017, pp. 505–10.

U.S. Geological Survey, Water Science School. The Distribution of Water on, in, and above the Earth. 2019, Accessed 22 Mar. 2026.

Wikipedia Contributors. Amazon sword. 2026, Accessed 23 Feb. 2026. en.wikipedia.org/wiki/Echinodorus_grisebachii.

Wikipedia Contributors. Anubias barteri. 2026, Accessed 23 Feb. 2026. en.wikipedia.org/wiki/Anubias_barteri.

Wikipedia Contributors. Siamese Fighting Fish. 2026, Accessed 23 Feb. 2026. en.wikipedia.org/wiki/Siamese_fighting_fish.

Wikipedia Contributors. The guppy. 2026, Accessed 23 Feb. 2026. en.wikipedia.org/wiki/Guppy.